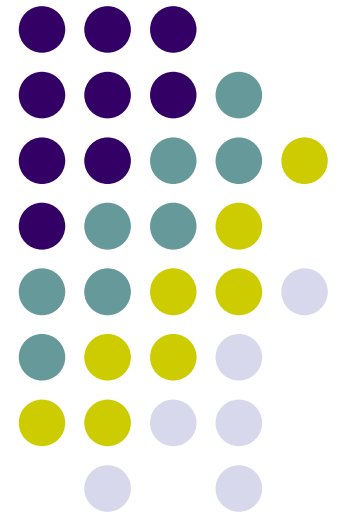
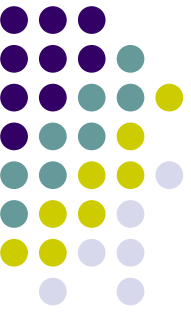


# Ch2 Linux 文件I/O

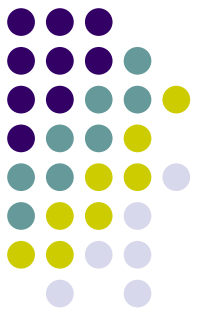
---





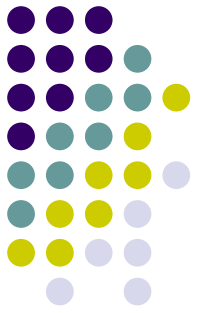
# 本章内容

- 文件属性 *ls -*
- 特殊文件
- 无缓冲文件I/O
- 缓冲文件I/O



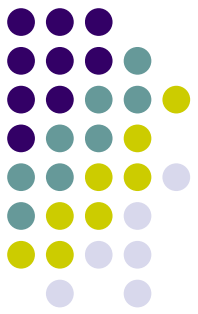
# 本章要求

- 理解文件属性的操作
- 熟练掌握文件存取权限的设置和掩码的作用
- 掌握目录、链接、设备特殊文件的处理方式
- 熟练掌握文件I/O函数的使用
- 理解文件描述符的操作
- 理解文件两种I/O控制方式的区别



# Part 1 文件属性

1. 文件属性
2. 文件类型
3. 文件存取权限
4. 其他属性



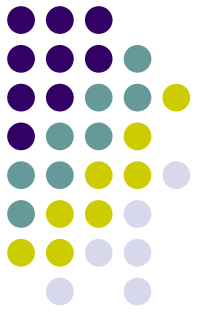
# 1.1 文件属性

## 1. 文件属性数据结构 **struct stat** —— 文件控制块

```
struct stat {  
    mode_t st_mode; /*file type & mode*/  
    ino_t st_ino; ls -li 索引能编号 /*inode number (serial number)*/  
    dev_t st_rdev; /*device number (file system)*/  
    nlink_t st_nlink; 链接数 /*link count*/ 硬链接计数  
    uid_t st_uid; /*user ID of owner*/  
    gid_t st_gid; /*group ID of owner*/  
    off_t st_size; 文件大小 /*size of file, in bytes*/  
    time_t st_atime; access 访问时 /*time of last access*/  
    time_t st_mtime; modify 修改时 /*time of last modification*/  
    time_t st_ctime; change 改变时 /*time of last file status change*/  
    long st_blksize; 块大小 /*Optimal block size for I/O*/  
    long st_blocks; 块数 /*number 512-byte blocks allocated*/  
};
```

命令查看: **ls -li file**

就是访问这个文件属性



## 2. stat/fstat/lstat 函数

- 获取文件属性

```
#include <sys/types.h>
```

```
#include <sys/stat.h>
```

```
#include <unistd.h>
```

获取文件属性  
`int stat(const char *file_name, struct stat *buf);`

文件描述符  
`int fstat(int filedes, struct stat *buf);`

lstat  
`int lstat(const char *file_name, struct stat *buf);`

(Return: 0 if success; -1 if failure)



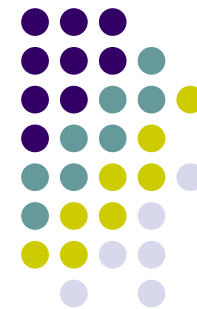
返回文件本身的属性。

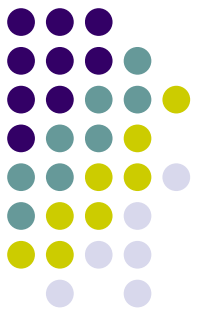
调用成功返回 0

失败返回 -1

## 1.2 文件类型

- 文件类型
- 获取文件类型

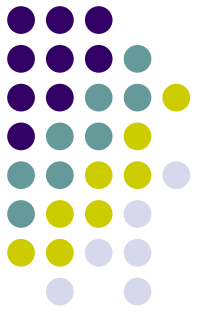




# 1. 文件类型

Unix/Linux系统支持的文件类型:

- Directory(d): 目录文件
- Link(l): 链接文件
- Pipe(p): 管道文件
- Block Device(b): 块设备文件
- Character Device(c): 字符设备文件
- Regular(-): 普通文件
- Socket(s): 套接字文件



# 1. 文件类型

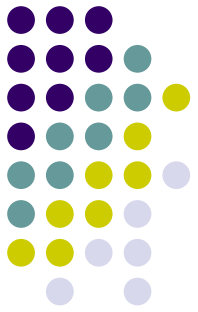
查看文件类型

- 使用命令：ls -l /dev/sda1

磁盘文件

可block

块设备



## 2. 获取文件类型

- `st_mode` 存储文件类型和许可权限，形式如下：

type3 type2 type1 type0 suid sgid sticky rwX rwX rwX

高性能文件类型      特殊权限      权限

- 用来确定文件类型的宏：

`S_ISBLK` -- 测试块文件

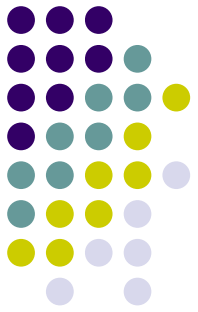
`S_ISCHR` -- 测试字符文件

`S_ISDIR` -- 测试目录

`S_ISFIFO` -- 测试FIFO

`S_ISREG` -- 测试普通文件 *regular*

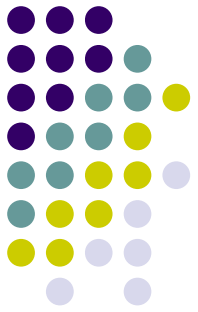
`S_ISLNK` -- 测试符号链接



# Example 2-2.c

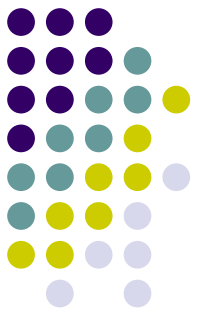
```
#include<stdio.h>
#include<sys/types.h>
#include<sys/stat.h>
int main(int argc,char *argv[])
{ int i; struct stat statbuf;
  for(i=1;i<argc;i++)
  { printf("%s:",argv[i]);
    if(lstat(argv[i],&statbuf)==-1) { printf("error\n"); continue; }
    if(S_ISDIR(statbuf.st_mode)) printf("%s is a directory\n",argv[i]);
    else if(S_ISREG(statbuf.st_mode)) printf("%s is a regular file\n",argv[i]);
    else if(S_ISBLK(statbuf.st_mode)) printf("%s is a block device\n",argv[i]);
    else if(S_ISCHR(statbuf.st_mode)) printf("%sis a charcter device\n",argv[i]);
    #ifdef S_ISLNK
    else if (S_ISLNK(statbuf.st_mode)) printf("%s is a link file\n",argv[i]);
    #endif
    #ifdef S_ISSOCK
    else if (S_ISSOCK(statbuf.st_mode)) printf("%s is a socket \n",argv[i]);
    #endif
    else printf("%s is an unknown type file \n",argv[i]);
  }
}
```

需要带参数运行  
例如 ./2-2 file



## 1.3 文件存取权限

- 文件存取权限
- 修改文件存取权限：命令/程序
- 默认文件存取权限：umask



# 1.文件存取权限

目录文件的权限说明：

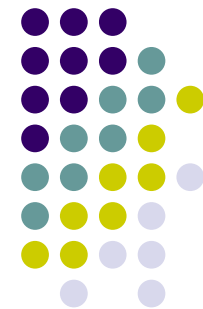
- 读的权限：显示目录文件，进入目录
- 写的权限：目录下创建文件
- 执行的权限：显示目录文件，进入目录，创建文件

ls命令：读和执行权限

cd命令：读和执行权限

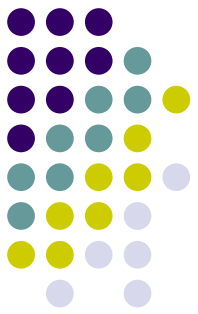
touch命令：写和执行权限

chmod - 可以改变或者修改



## 2. 改变文件存取权限——程序

- 例2.3 设计一个程序，要求把系统中“/home/mylinux”目录下的myfile文件权限，设置成文件所有者可读可写，其他用户只读权限。



## Example 2-3.c

```
#include<sys/types.h>
```

```
#include<sys/stat.h>
```

```
int main ()
```

```
{
```

```
chmod("/home/mylinux/myfile", S_IRUSR|S_I  
WUSR|S_IRGRP|S_IROTH);
```

```
return 0;
```

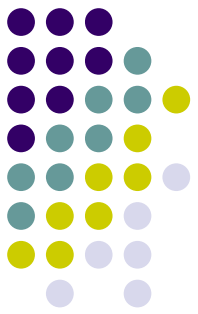
```
}
```

可读

可读

644

可执行



# chmod 函数

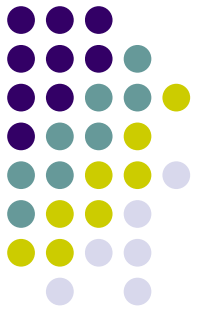
- Change permissions of a file

```
#include <sys/types.h>
```

```
#include <sys/stat.h>
```

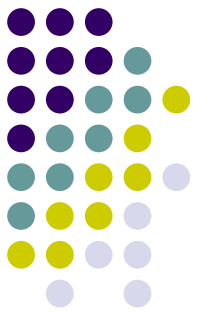
```
int chmod(const char *path, mode_t mode);
```

```
int fchmod(int fildes, mode_t mode); // 打开的文件  
(Return: 0 if success; -1 if failure)
```



## 2. 改变文件存取权限

| st_mode屏蔽      | 含义                                |
|----------------|-----------------------------------|
| S_IRUSR(00400) | Read by owner                     |
| S_IWUSR(00200) | Write by owner                    |
| S_IXUSR(00100) | Execute by owner                  |
| S_IRWXU(00700) | Read, write and execute by owner  |
| S_IRGRP(00040) | Read by group                     |
| S_IWGRP(00020) | Write by group                    |
| S_IXGRP(00010) | Execute by group                  |
| S_IRWXG(00070) | Read, write and execute by group  |
| S_IROTH(00004) | Read by others                    |
| S_IWOTH(00002) | Write by others                   |
| S_IXOTH(00001) | Execute by others                 |
| S_IRWXO(00007) | Read, write and execute by others |

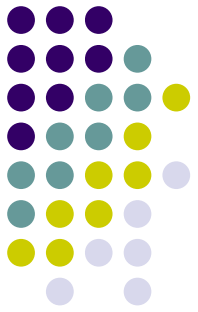


### 3. 默认文件存取权限——umask

- The default permission for newly created files and directories are(root):
  - File: -rw-r--r-- 644
  - Directory: drwxr-xr-x 755
- Why?
  - umask: an octal number which contains the bits that are NOT set as permission bits.
  - Default 002 for normal user, 022 for root.

普通用户 002

root 022

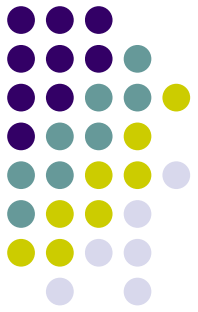


# umask 函数

- 例2.4 设计一程序，要求设置系统文件和目录的权限掩码。
- 注意权限掩码影响范围

① umask 从当前值开始

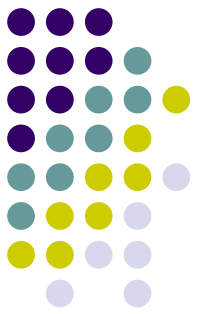
② 程序中：从本次设置生效，到下次设置前



# Example 2-4.c

```
#include<stdio.h>
#include<stdlib.h>
#include<sys/stat.h>
#include<sys/types.h>
int main()
{   mode_t new_umask,old_umask;
    new_umask=0666;
    old_umask=umask(new_umask);
    printf("系统原来的权限掩码是: %o\n",old_umask);
    printf("系统新的权限掩码是: %o\n",new_umask);
    system("touch liu1");
    printf("创建了文件liu1\n");
    new_umask=0444;
    old_umask=umask(new umask);
    printf("系统原来的权限掩码是: %o\n",old_umask);
    printf("系统新的权限掩码是: %o\n",new_umask);
    system("touch liu2");
    printf("创建了文件liu2\n");
    system("ls liu1 liu2 -l");
    return 0;
}
```

程序执行后权限与执行前相同



# umask 函数

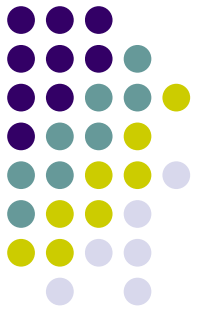
- 为进程设置文件存取权限屏蔽字，并返回以前的值

```
#include <sys/types.h>
```

```
#include <sys/stat.h>
```

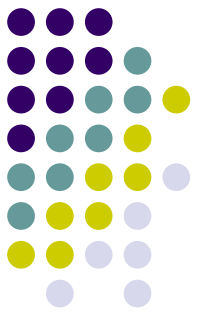
```
mode_t umask(mode_t mask);
```

- 建立文件，文件的权限为 $0666 - \text{mask}$ ；  
建立目录，目录权限为 $0777 - \text{mask}$ 。



## 1.4 文件其他属性

- 文件主 <sup>文件主</sup> chown 系列函数
- 文件存取时间 stat 函数
- 文件大小 stat 函数



# 1.chown/fchown/lchown 函数

- Change ownership of a file 改变文件所有者

```
#include <sys/types.h>
```

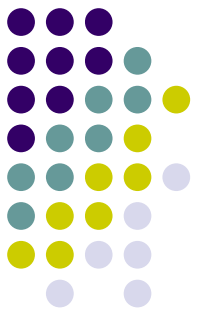
```
#include <unistd.h>
```

```
int chown(const char *path, uid_t owner, gid_t group);
```

```
int fchown(int fd, uid_t owner, gid_t group); 打开文件
```

```
int lchown(const char *path, uid_t owner, gid_t group);
```

```
(Return: 0 if success; -1 if failure)
```

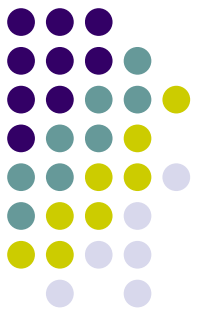


## 2. 获取文件存取时间——stat

### Example 2-5.c

```
#include<stdio.h>
#include<sys/stat.h>
#include<time.h>
main()
{
    char *path="./2-5.c";
    struct stat statbuf;
    if (stat(path,&statbuf)==-1)
        perror("Failed to get file status");
    else printf("%s last accessed at %s",path,ctime(&statbuf.st_atime));
}
```

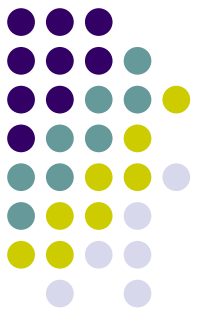
获取文件时间...



## 3. 获取文件大小——stat

### Example 2-6.c

```
#include<stdio.h>
#include<unistd.h>
#include<sys/stat.h>
int main ()
{
    struct stat buf;
    stat("/etc/passwd",&buf);
    printf("/etc/passwd文件的大小是: %d\n",buf.st_size);
    return 0;
}
```

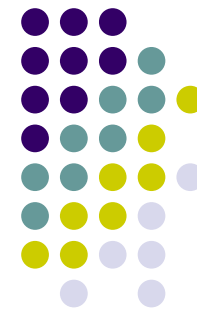


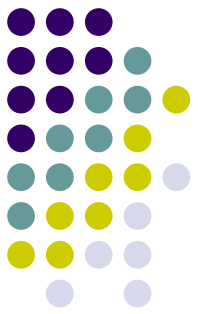
# 掌握的知识点

- 文件属性函数: `stat`, `lstat`, `fstat`
- 常见的文件类型
- 文件权限的表示方法
- 修改文件权限
- 设置默认文件权限
- 改变文件主 `chown`
- 文件存取时间 `stat` -> `st_atime`
- 文件大小 `stat` -> `st_size`

# Part 2 特殊文件

- 目录文件
- 链接文件
- 设备文件

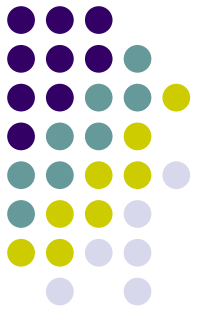




## 2.1 目录文件 *→ 目录结构*

- mkdir/rmdir
- *cd* chdir/fchdir, *pwd* getcwd
- 读目录操作
  - opendir/closedir
  - readdir
  - telldir
  - seekdir

例2.7设计一程序，要求读取当前目录文件中所有的目录结构。



# Example 2-7.c

```
#include<sys/types.h>
#include<dirent.h>
#include<unistd.h>
#include<stdio.h>
int main()
```

```
{
```

```
    DIR * dir;
```

```
    struct dirent * ptr;
```

```
    int i;
```

```
    dir=opendir("./");
```

```
    while((ptr = readdir(dir))!=NULL)
```

```
    {
```

```
        printf("目录: %s\n", ptr->d_name);
```

```
    }
```

```
    closedir(dir);
```

```
}
```

当前目录 .

⇒ 打开当前目录

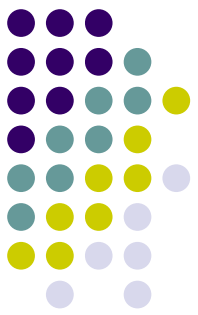
结构体

结构体

d\_name

inode 系统编号

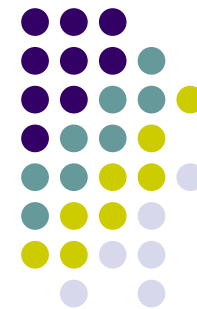
inode



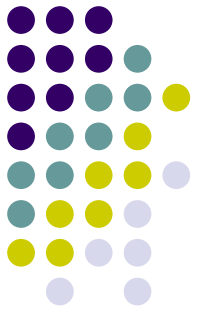
# 运行结果

```
mylinux@mylinux-desktop: ~/code/ch2
File Edit View Terminal Tabs Help
mylinux@mylinux-desktop: ~/code/ch2$ gcc -o 2-14 2-14.c -g
mylinux@mylinux-desktop: ~/code/ch2$ ./2-14
目录: .
目录: ..
目录: 2-1_file_type.c
目录: 2-10.c
目录: 2-11.c
目录: 2-13.c
目录: 2-14.c
目录: 2-15.c
目录: 2-16.c
目录: 2-2_file_mode.c
目录: 2-3_umask.c
目录: 2-4_stat.c
目录: 2-5.c~
目录: 2-6.c
目录: 2-7.c
目录: 2-8.c
目录: 2-9.c
目录: 2-12
目录: 2-1
目录: 2-2
目录: 2-1_file_type.c~
目录: 2-3
```

# 读目录



- 数据结构
  - DIR, struct dirent
- 操作函数
  - opendir/closedir
  - readdir
  - telldir
  - seekdir



# 数据结构

- DIR

- 目录流对象的数据结构
- in <dirent.h>

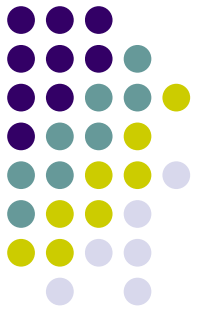
typedef struct \_\_dirstream DIR;

- struct dirent

- 目录项
- Defined in <dirent.h>

ino\_t d\_ino; /\* inode number \*/  
char d\_name[NAME\_MAX + 1]; /\* file name \*/

文件名



# 操作函数

- 目录的打开、关闭、读、定位

```
#include <sys/types.h>
```

```
#include <dirent.h>
```

```
DIR *opendir(const char *name);
```

```
int closedir(DIR *dir);
```

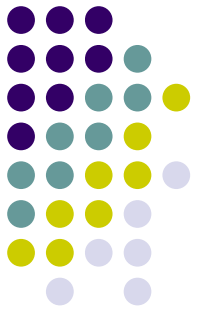
```
struct dirent *readdir(DIR *dir);
```

```
off_t telldir(DIR *dir);
```

获取当前位置

```
void seekdir(DIR *dir, off_t offset);
```

设置指针位置



# 目录扫描程序——ls -R命令

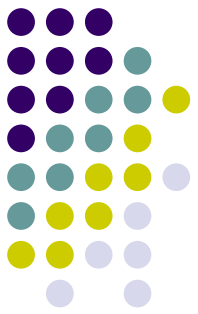
```
DIR *dp;  
struct dirent *entry;
```

```
if ( (dp = opendir(dir)) == NULL )  
    err_sys(...);  
while ( (entry = readdir(dp)) != NULL ) {  
    lstat(entry->d_name, &statbuf);  
    if ( S_ISDIR(statbuf.st_mode) )  
        ...  
    else  
        ...  
}  
closedir(dp);
```

递归扫描  
每个子目录

是目录!

不是目录



# mkdir/rmdir 函数

- 创建一个空目录

```
#include <sys/stat.h>
```

```
#include <sys/types.h>
```

```
int mkdir(const char *pathname, mode_t mode);
```

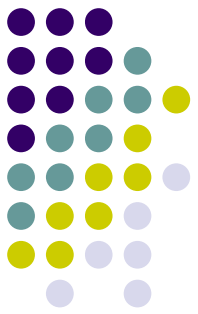
(Return: 0 if success; -1 if failure)

- 删除一个空目录

```
#include <unistd.h>
```

```
int rmdir(const char *pathname);
```

(Return: 0 if success; -1 if failure)



# chdir/fchdir 函数

- Change working directory

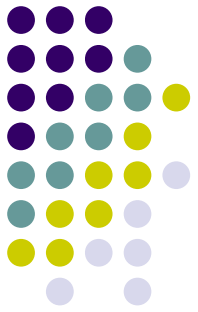
```
#include <unistd.h>
```

```
int chdir(const char *path);
```

```
int fchdir(int fd);
```

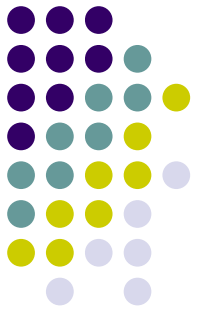
(Return: 0 if success; -1 if failure)

- 当前工作目录是进程的属性，所以该函数只影响调用chdir的进程本身
  - cd(1) command



## 2.2 链接文件

- In 命令 *硬链接*
- link/unlink 函数
  - ❖ 给一个文件创建一个链接.
    - ❖ `#include <unistd.h>`
    - ❖ `int link(const char *oldpath, const char *newpath);`
    - ❖ (Return: 0 if success; -1 if failure)
  - ❖ 删除文件链接
    - ❖ `#include <unistd.h>`
    - ❖ `int unlink(const char *pathname);`
    - ❖ (Return: 0 if success; -1 if failure)



# symlink/readlink

符号链接

- ln -s命令
- 创建一个符号链接

建一个文件, 存储它的名字  
link

`#include <unistd.h>`

`int symlink(const char *oldpath, const char *newpath);`

(Return: 0 if success; -1 if failure)

- 读取符号链接的值

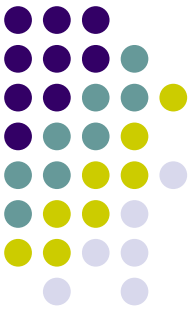
`#include <unistd.h>`

读链接

`int readlink(const char *path, char *buf, size_t bufsiz);`

(Return: the count of characters placed in the buffer if success; -1 if failure)

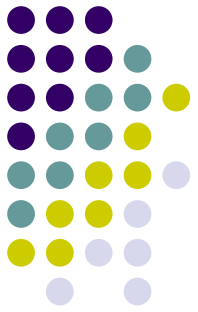
例2.8设计一程序, 要求为 “/etc/passwd” 文件建立软链接 “2-8link”, 并查看此链接文件和 “/etc/passwd” 文件。



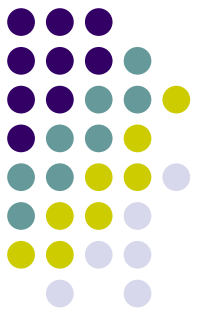
# Example 2-8link.c

```
#include<unistd.h>
int main()
{
    symlink("/etc/passwd","2-8link");
    system("ls 2-8link -l");
    system("ls /etc/passwd -l");
}
```

# Example 2-9 link.c

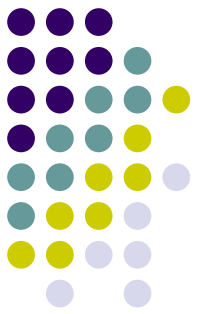


例2.9设计一程序，要求为“/etc/passwd”文件建立硬链接“2-9link”，并查看此链接文件和“/etc/passwd”文件。



## Example 2-95link.c

```
#include<unistd.h>
int main()
{
    link("./myfile","2-9link");
    system("ls 2-9link -l");
    system("ls /etc/passwd -l");
}
```



## 2.3 设备文件

- 1.设备文件名

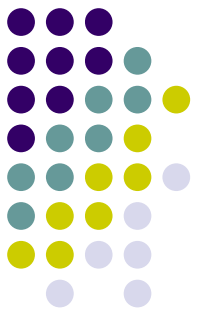
`ls -C` 列出当前系统加载的设备对应的文件

`ls -li` 列出当前终端设备的属性

- 2.设备文件读写

`open, read, write, close, stat`

例2-10 向终端pts1写入100个“unix”。



## Example2-10

- `#include<fcntl.h>`

```
int main()
```

```
{ int i,fd;
```

```
    fd=open("/dev/pts/1",O_WRONLY);
```

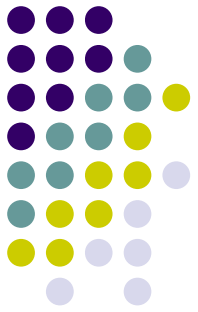
```
    for(i=0;i<100;i++)
```

```
        write(fd, "Unix",4);
```

```
    close(fd);
```

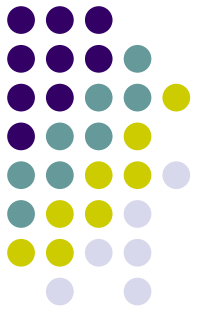
```
}
```

打开两个**terminal**，一个运行程序，一个输入**tty**，观察运行结果



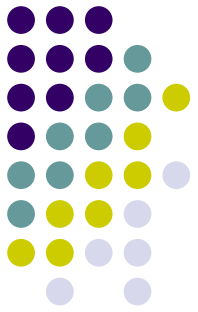
# 掌握的知识点

- 目录的读写流程
- 链接文件和设备文件的基本操作



## Part 3 无缓冲文件I/O

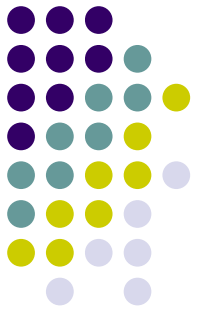
- 两种I/O方式
- 文件描述符
- 无缓冲I/O函数：  
`creat, open, close, read/write, lseek, dup/dup2`



## 3.1 两种I/O方式

### 1. 无缓冲和缓冲I/O

- 无缓冲I/O
  - read/write -> 系统函数
  - 文件描述符
  - POSIX.1 and XPG3标准
- 缓冲 I/O
  - 标准I/O库实现
  - 处理很多细节, 如缓存分配, 以优化长度执行I/O等.
  - 流 -> FILE类型指针

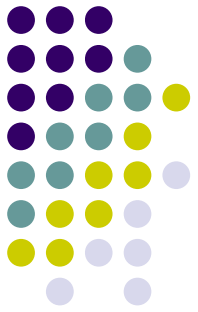


## 3.1 两种I/O方式

### 2. 无缓冲 I/O 系统调用

- 基本 I/O

- open/creat, close, read, write, lseek
- dup/dup2
- fcntl *fcntl*

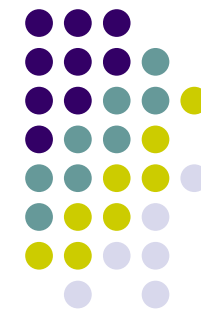


## 3.2 文件描述符

### 1. 文件描述符概述

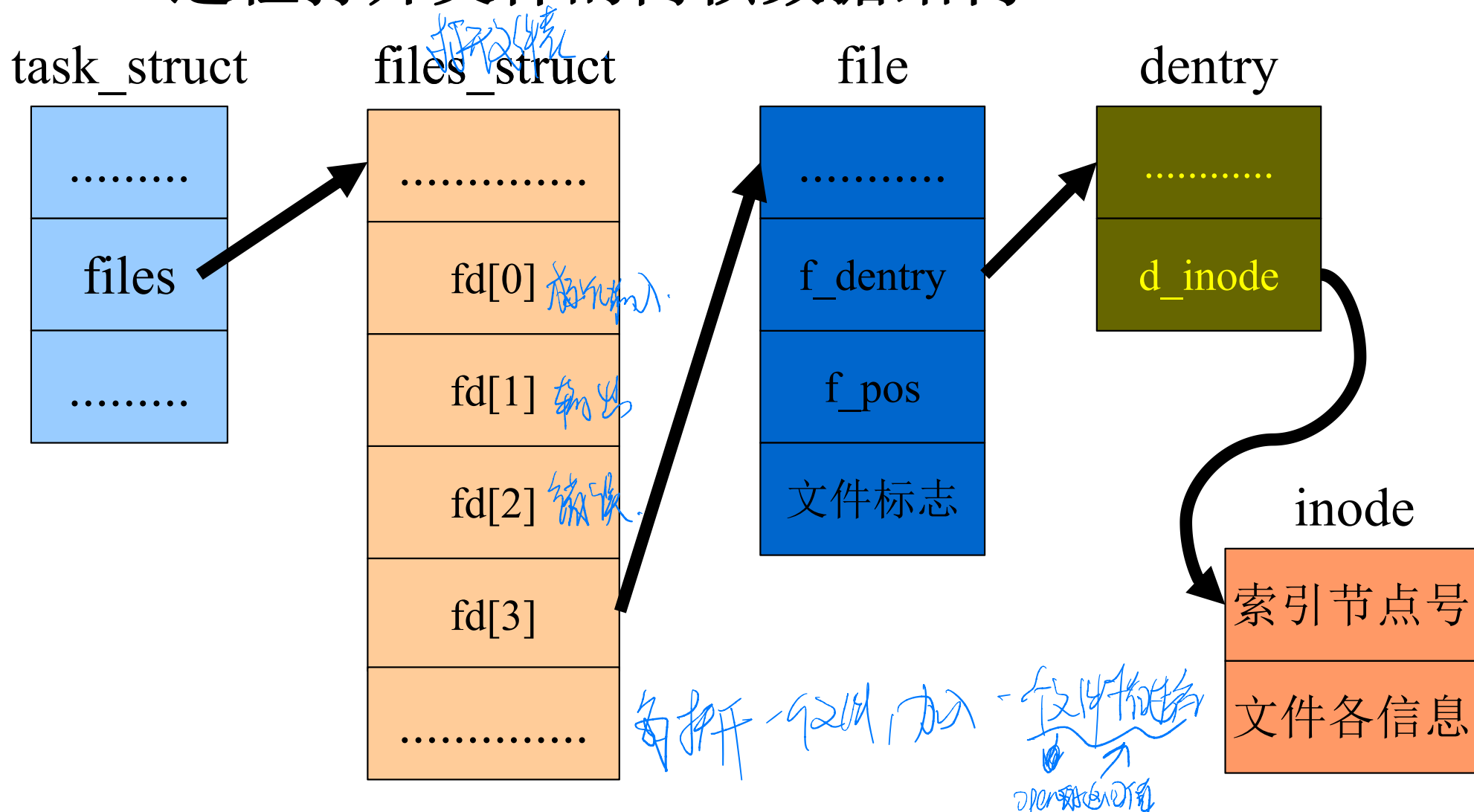
- 非负整数
  - `int fd;`
- (in `<unistd.h>`)
- `STDIN_FILENO (0), STDOUT_FILENO (1), STDERR_FILENO (2)`
- 文件操作一般步骤:  
`open-read/write-[lseek]-close`

总结



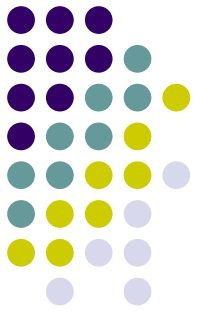
## 3.2 文件描述符

- 2. 进程打开文件的内核数据结构



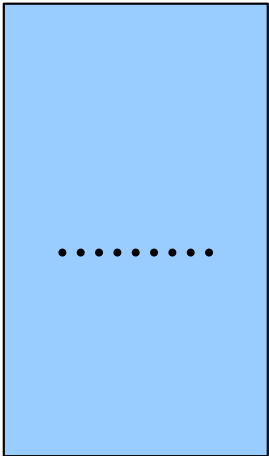
## 3.2 文件描述符

### 2. 进程打开文件的内核数据结构



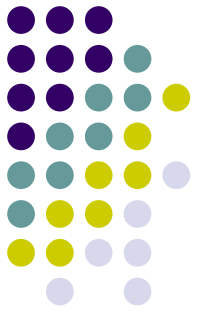
task\_struct

进程控制块  
PCB

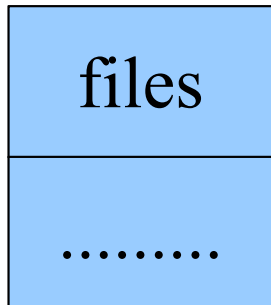


## 3.2 文件描述符

### 2. 进程打开文件的内核数据结构



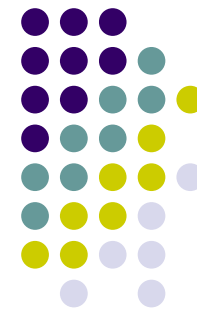
task\_struct



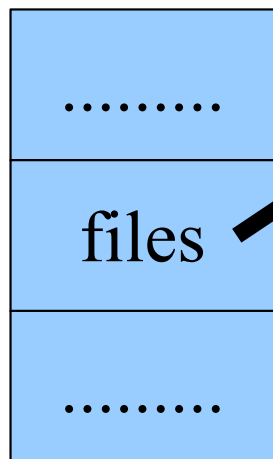
```
struct task_struct{  
    .....  
    struct files_struct *files;  
    .....  
};
```

## 3.2 文件描述符

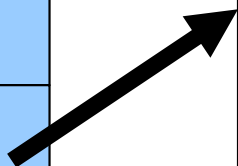
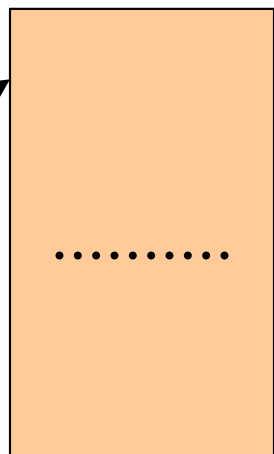
### 2. 进程打开文件的内核数据结构



task\_struct



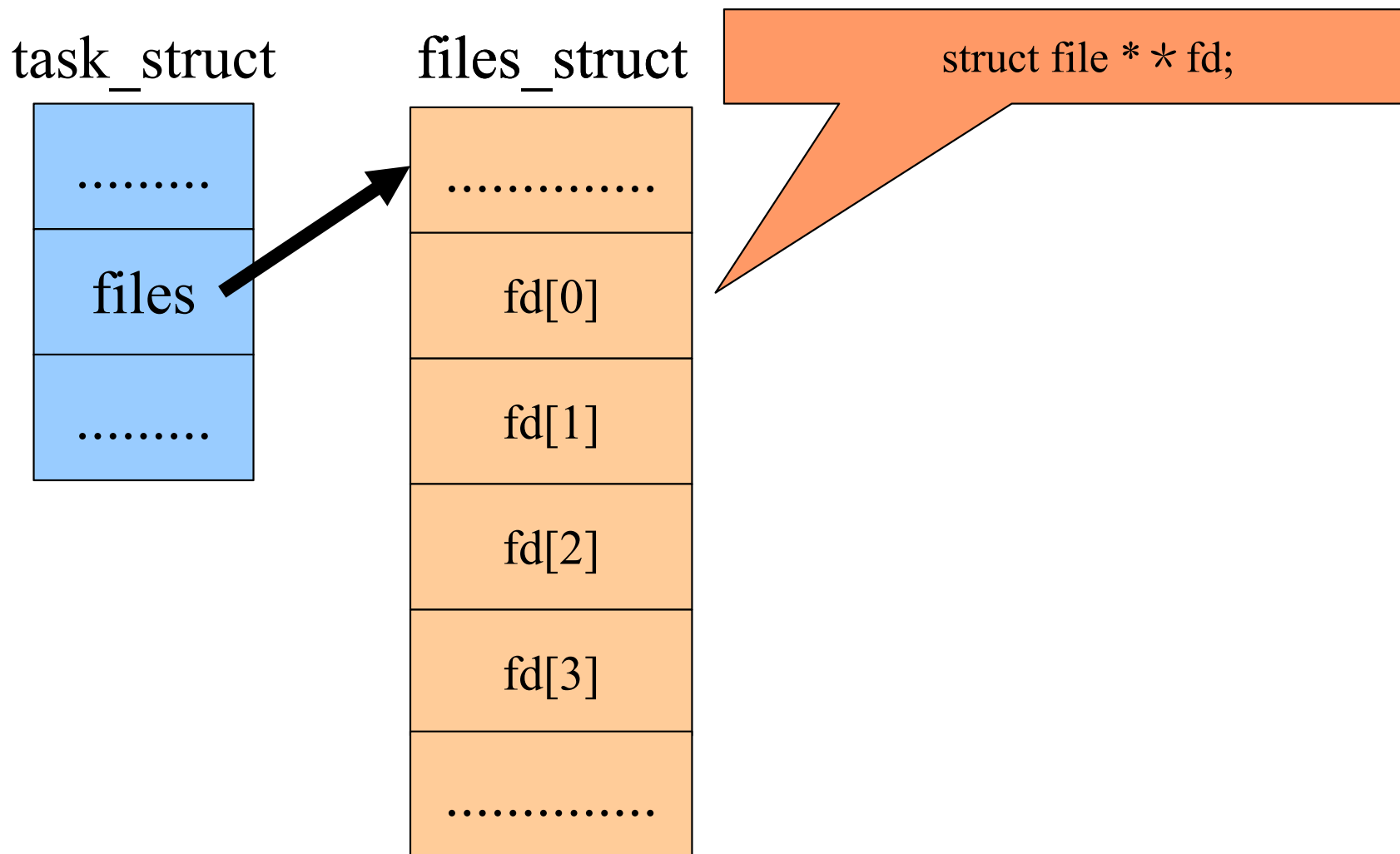
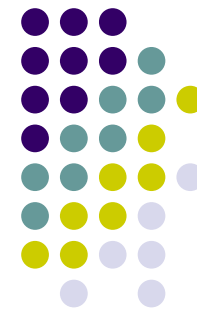
files\_struct



该结构体包含了：  
进程已打开文件表

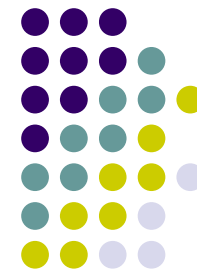
## 3.2 文件描述符

### 2. 进程打开文件的内核数据结构

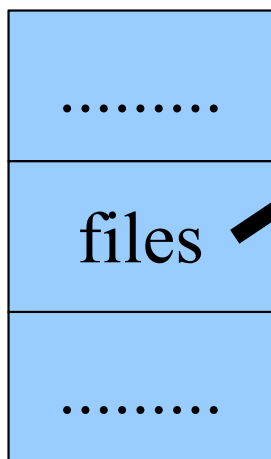


## 3.2 文件描述符

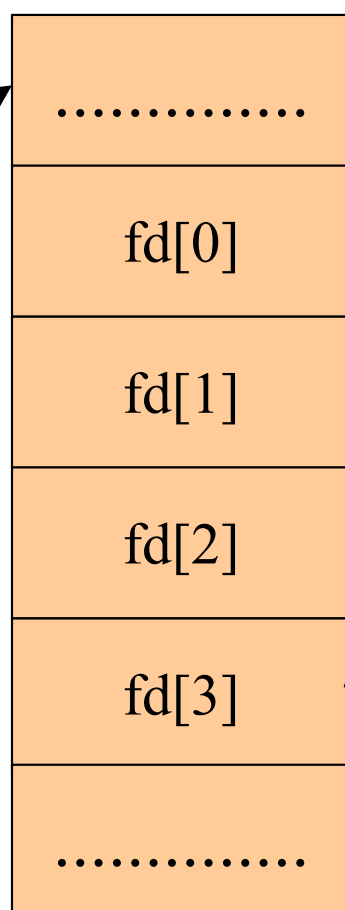
### 2. 进程打开文件的内核数据结构



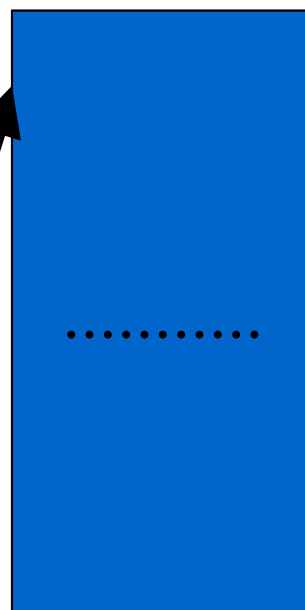
task\_struct



files\_struct



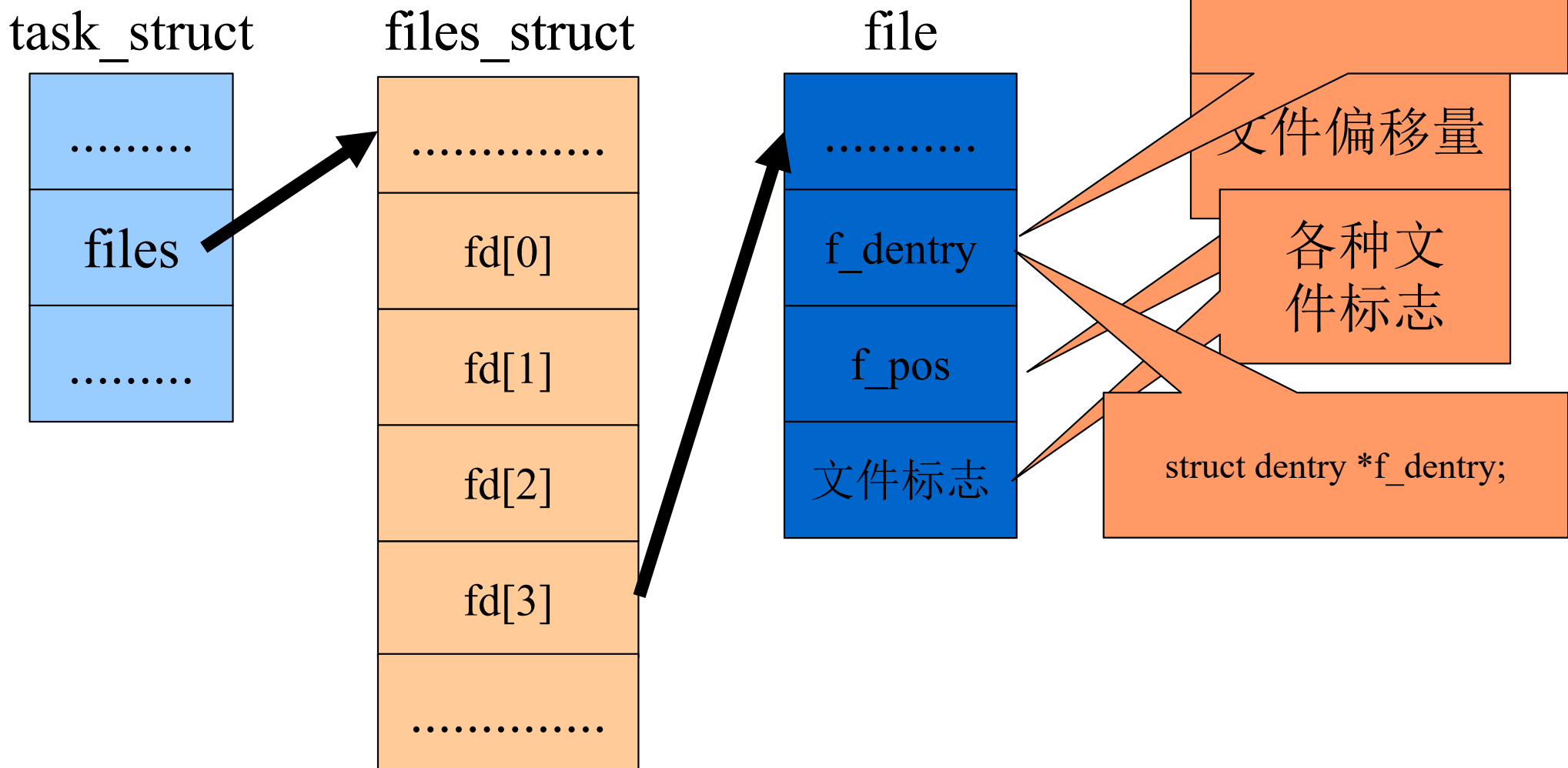
file



文件对象：代表一个已打开的文件

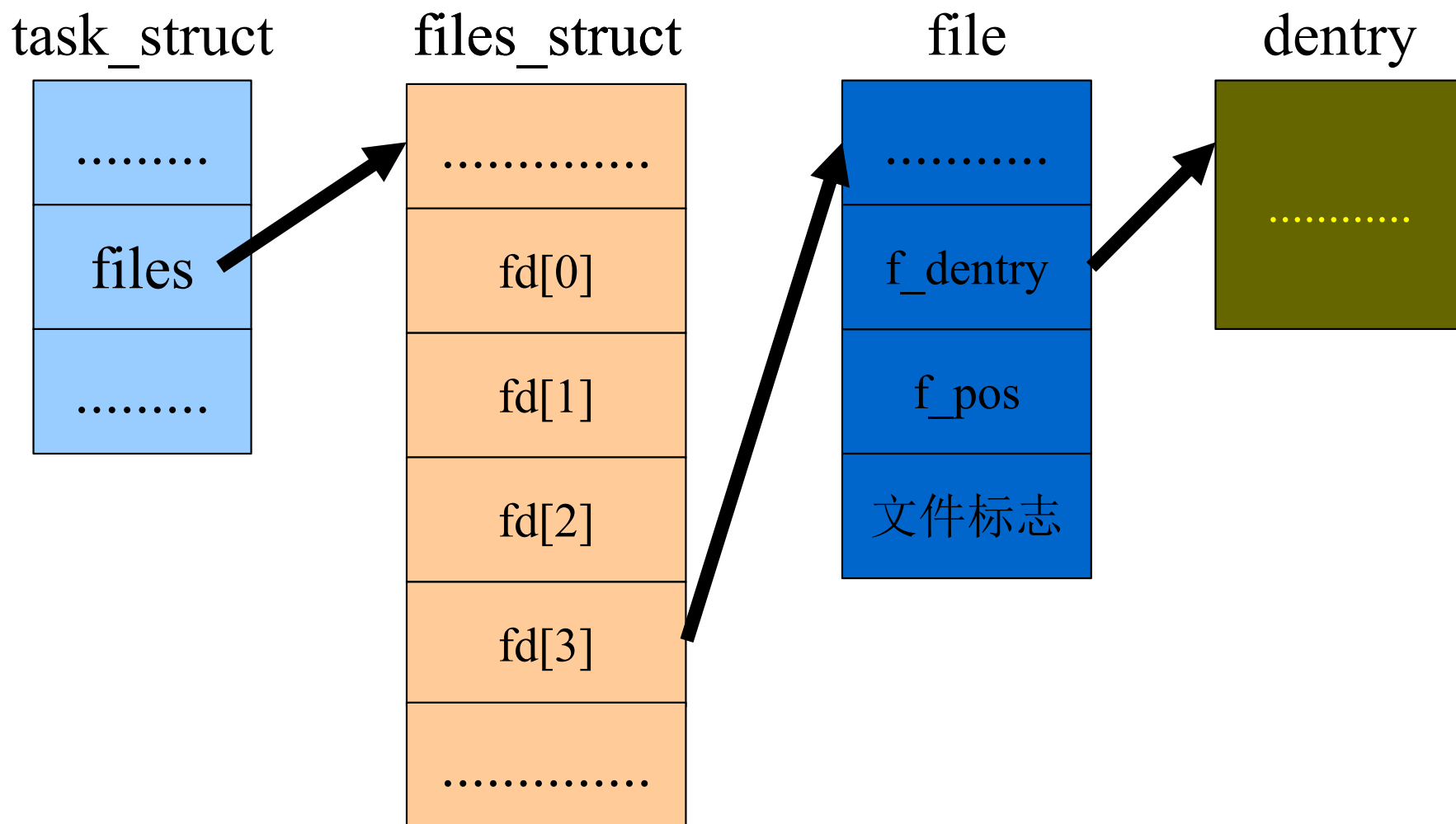
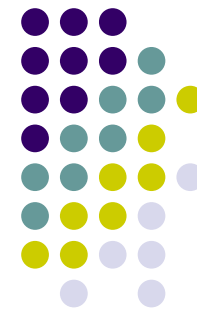
## 3.2 文件描述符

### 2. 进程打开文件的内核数据结构



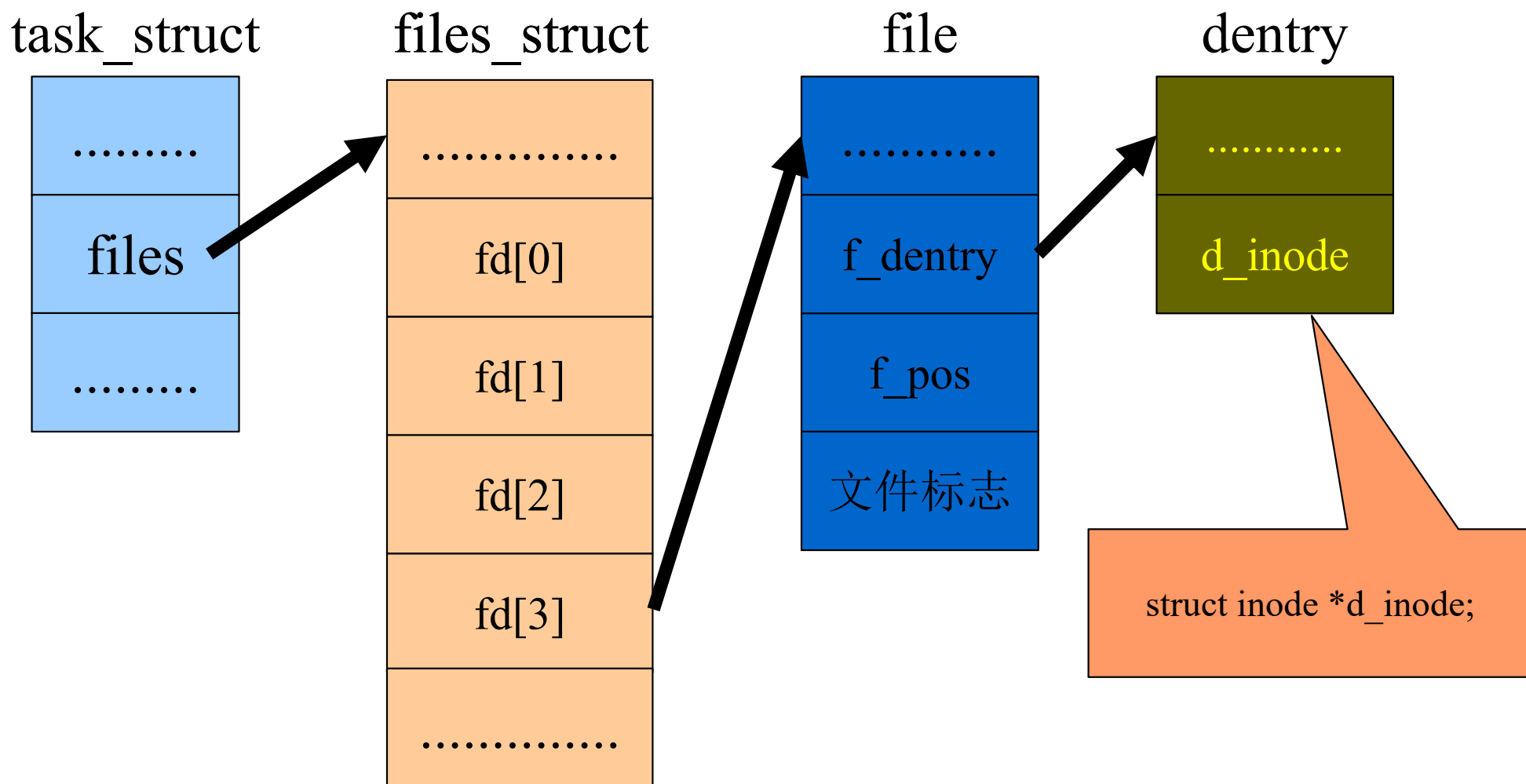
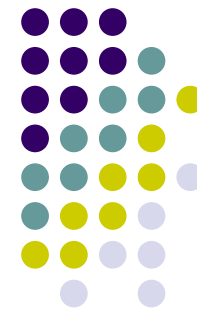
## 3.2 文件描述符

### 2. 进程打开文件的内核数据结构



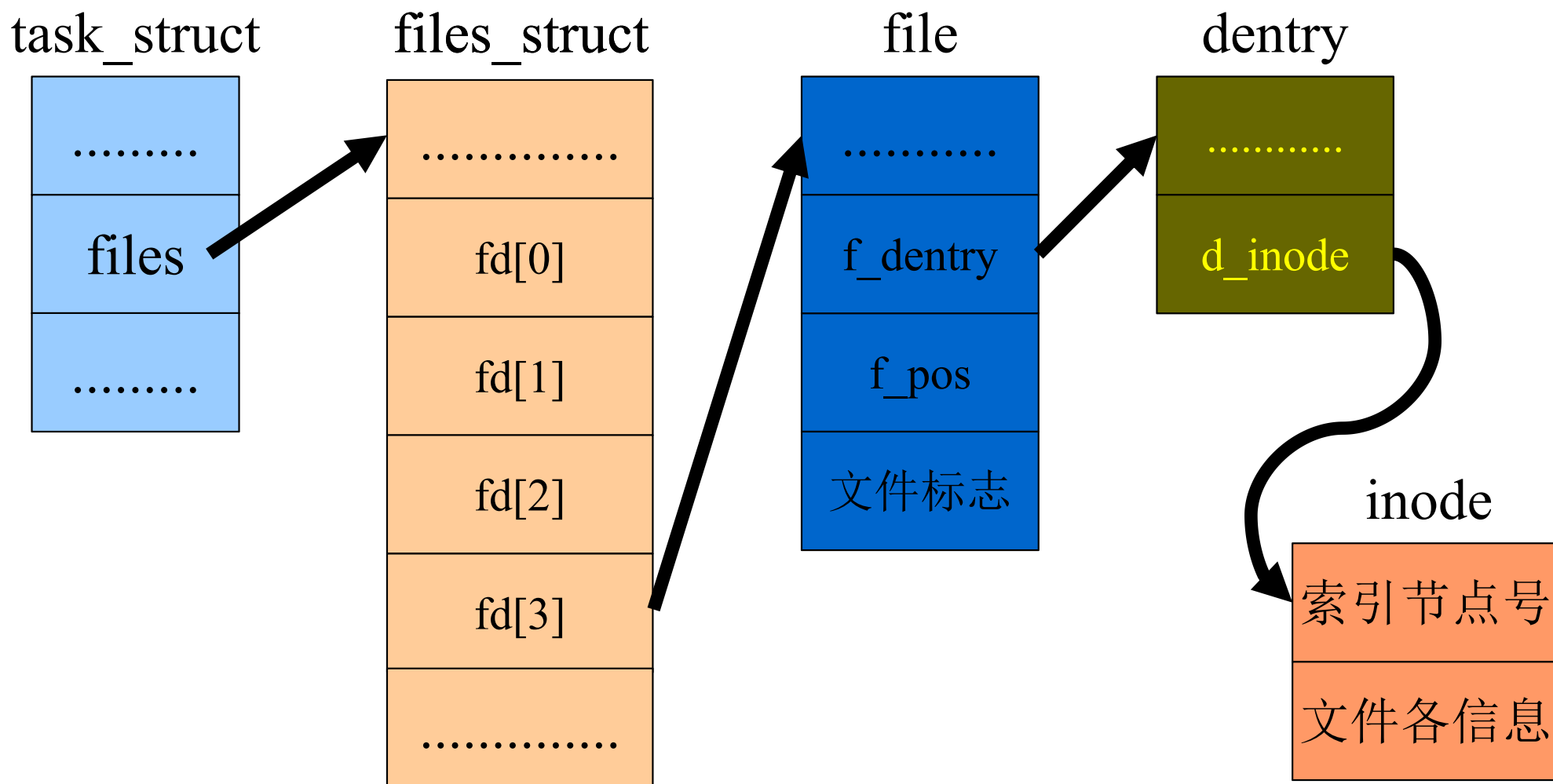
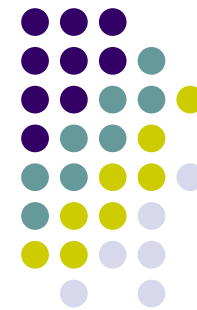
## 3.2 文件描述符

### 2. 进程打开文件的内核数据结构



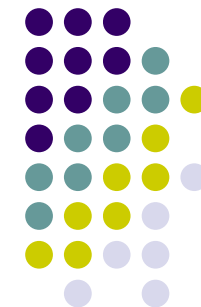
## 3.2 文件描述符

### 2. 进程打开文件的内核数据结构

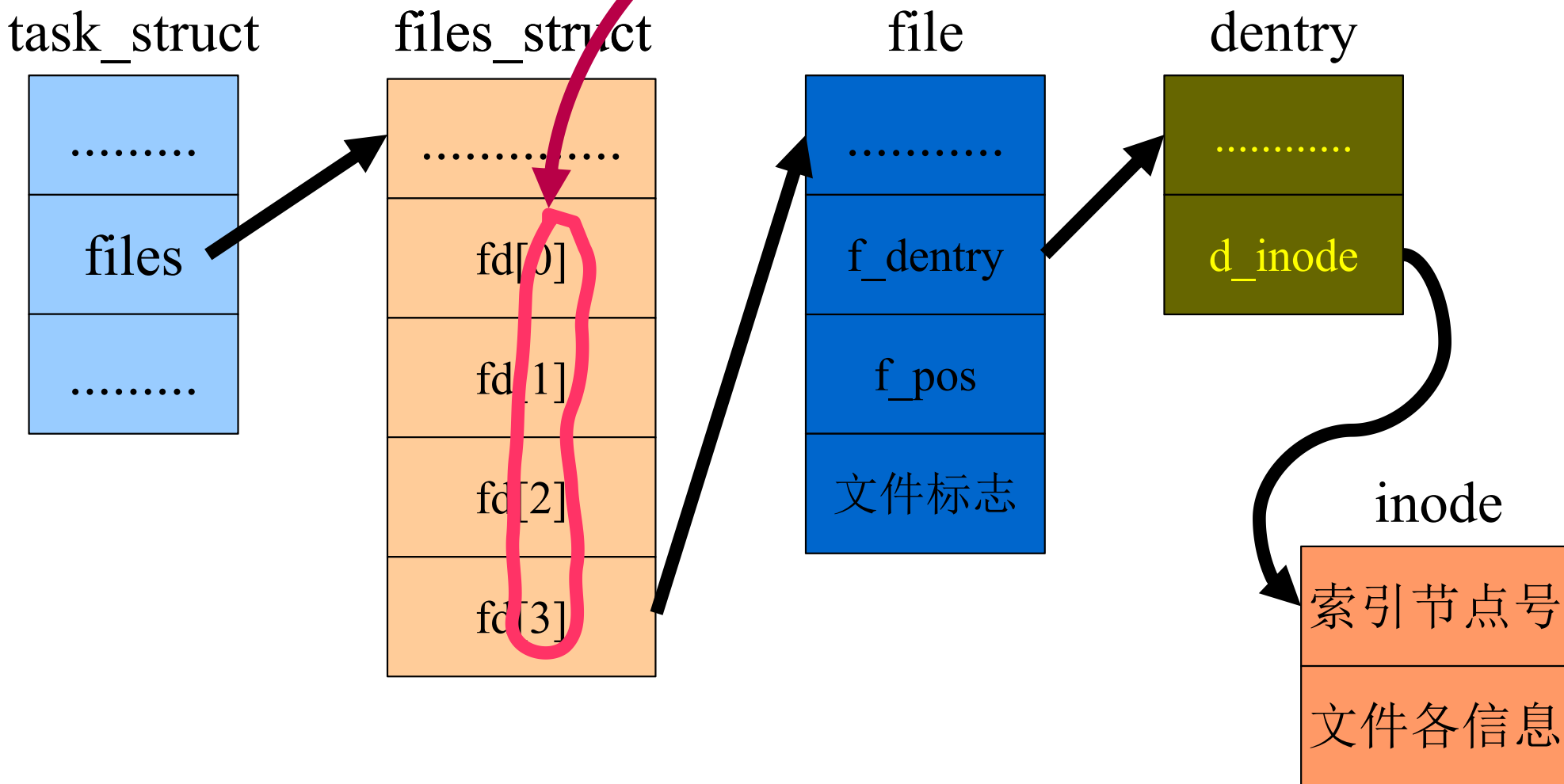


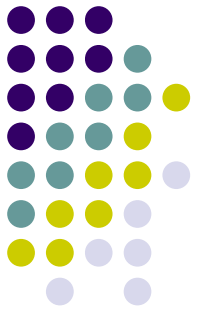
# 3.2 文件描述符

## 2. 进程打开文件的内核数据结构



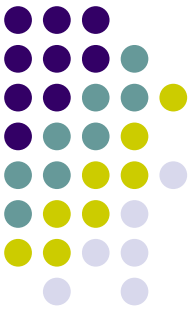
open函数返回的文件描述符  
已打开文件表的索引





## 3.3 无缓冲 I/O函数

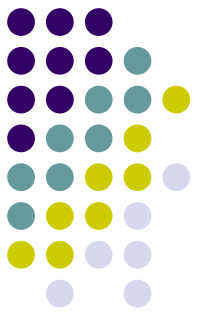
- open/creat, close, read, write, lseek
- dup/dup2
- fcntl
- ioctl



# 0. 错误处理

- UNIX方式
  - Return value
  - “errno”变量( defined in /usr/include/errno.h)  
extern int errno;
- strerror & perror

```
#include <string.h>
char *strerror(int errnum);
#include <stdio.h>
void perror(const char *msg);
```



# 1. creat 函数

create a file or device

```
#include <sys/types.h>
```

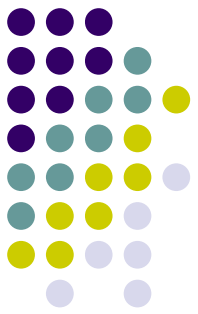
```
#include <sys/stat.h>
```

```
#include <fcntl.h>
```

```
int creat(const char *pathname, mode_t mode);
```

(Return: a new file descriptor if success; -1 if failure)

**例2.11** 设计一程序，要求在“/home”目录下创建一个名称为“2-7file”的文件，并且把此文件的权限设置为所有者具有只读权限，最后显示此文件的信息。

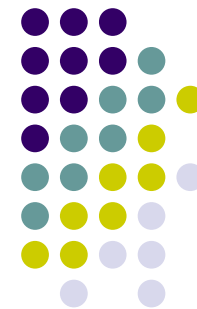


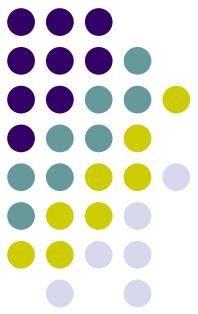
# Example2-11.c

```
#include<sys/types.h>
#include<sys/stat.h>
#include<fcntl.h>
int main()
{
    int fd;
    fd=creat("./2-11file",S_IRUSR);
    system("ls ./2-11file -l");
    return 0;
}
```

# 参数 “mode”

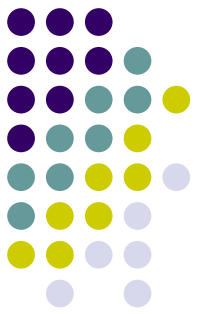
- “mode”: 指定创建的新文件的存取权限





# 参数 “mode”取值

| 取值             | 含义                                |
|----------------|-----------------------------------|
| S_IRUSR(00400) | Read by owner                     |
| S_IWUSR(00200) | Write by owner                    |
| S_IXUSR(00100) | Execute by owner                  |
| S_IRWXU(00700) | Read, write and execute by owner  |
| S_IRGRP 00040  | Read by group                     |
| S_IWGRP 00020  | Write by group                    |
| S_IXGRP 00010  | Execute by group                  |
| S_IRWXG 00070  | Read, write and execute by group  |
| S_IROTH 00004  | Read by others                    |
| S_IWOTH 00002  | Write by others                   |
| S_IXOTH 00001  | Execute by others                 |
| S_IRWXO 00007  | Read, write and execute by others |



# 参数 “mode” & umask

- umask: 一种文件保护机制
- 新建文件的初始存取权限
  - mode & ~umask

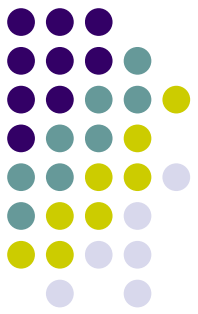
regular files:

|                       |           |     |
|-----------------------|-----------|-----|
| default permissions   | rw-rw-rw- | 666 |
| umask ( - )           | ---w--w-  | 022 |
| resulting permissions | rw-r--r-- | 644 |

523.1  
取反

directories:

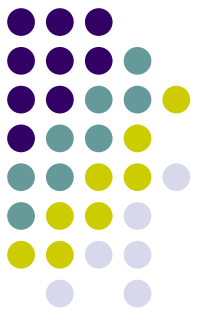
|                       |           |     |
|-----------------------|-----------|-----|
| default permissions   | rw-rwxrwx | 777 |
| umask ( - )           | ---w--w-  | 022 |
| resulting permissions | rw-r-xr-x | 755 |



# 课堂练习

- 假设系统的umask为0022，执行  
`creat("a.txt",454)`后，文件a.txt的所有者、同  
组用户、其他用户对文件的权限是什么？

444



## 2. Open 函数

- Open and possibly create a file or device

```
#include <sys/types.h>
```

```
#include <sys/stat.h>
```

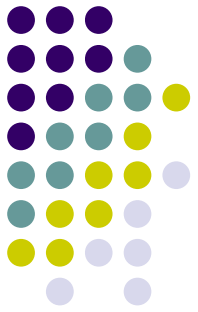
```
#include <fcntl.h>
```

```
int open(const char *pathname, int flags);
```

```
int open(const char *pathname, int flags, mode_t mode);
```

(Return: a new file descriptor if success; -1 if failure)

例2.8设计一个程序，要求在当前目录下以可读写方式打开一个名为“**2-8file**”的文件。如果该文件不存在，则创建此文件；如果存在，将文件清空后关闭。



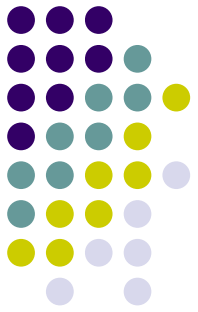
# Example2-12.c

```
#include<stdio.h>
#include<stdlib.h>
#include<fcntl.h>
int main ()
{   int fd;
    if(fd=open("./2-12file",O_CREAT|O_TRUNC|O_WRONLY,0600))<0)
    {   perror("打开文件出错");
        exit(1);
    }
    else
    {   printf("打开(创建)文件\"2-8file\", 文件描述符为: %d\n",fd);
    }
    if(close(fd)<0)
    {   perror("关闭文件出错");
        exit(1);
    }
    system("ls ./2-12file -l");
    return 0;
}
```

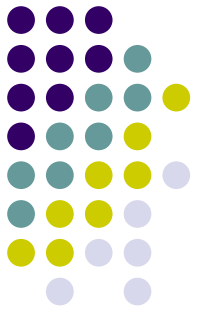
*Handwritten annotations:*

- open* (with an arrow pointing to the `open` function call)
- 创建* (under `O_CREAT`)
- 清空* (under `O_TRUNC`)
- 写权限* (under `O_WRONLY`)

# 参数 “flags”



- “flags”: 指定文件存取方式
  - One of **O\_RDONLY**, **O\_WRONLY** or **O\_RDWR** which request opening the file read-only, write-only or read/write, respectively, bitwise-or'd with zero or more of the following: ( All defined in /usr/include/fcntl.h)
    - O\_APPEND**: 追加方式打开
    - O\_TRUNC**: 文件存在清空
    - O\_CREAT**: 文件不存在创建.
    - O\_EXCL**: 和 **O\_CREAT**一起用时, 文件存在则出错, 打开失败
    - ...
- “creat” function: 等价于open函数中指定 **O\_CREAT|O\_WRONLY|O\_TRUNC**

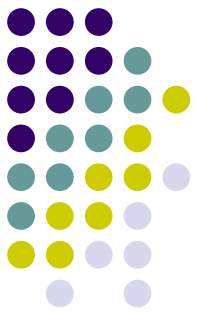


### 3. close 函数

- Close a file descriptor

#include <unistd.h>

int close(int fd); *关闭一个文件描述符*  
(Return: 0 if success; -1 if failure)



## 4. read/write 函数 *读/写*

- Read from a file descriptor

`#include <unistd.h>`

`ssize_t read(int fd, void *buf, size_t count);`

(返回值: 读到的字节数, 若已到文件尾为0, 若出错为-1)

- Write to a file descriptor

`#include <unistd.h>`

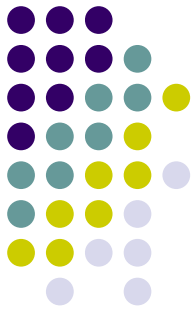
`ssize_t write(int fd, const void *buf, size_t count);`

(返回值: 若成功为已写的字节数, 若出错为-1)

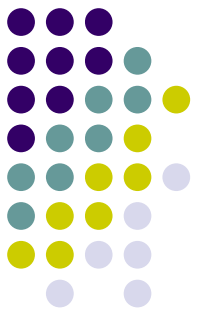
**例2.13**设计一个程序, 完成文件的复制工作。要求通过**read**函数和**write**函数复制“**/etc/passwd**”文件到目标文件中, 目标文件名在程序运行时从键盘输入。

# Example2-13.c

CP.



```
#include<unistd.h>
#include<sys/types.h>
#include<sys/stat.h>
#include<fcntl.h>
#include<stdio.h>
#include<stdlib.h>
int main()
{
    int fdsrc,fddes,nbytes; int z; char buf[20], des[20];
    int flags=O_CREAT | O_TRUNC | O_WRONLY;
    printf("请输入目标文件名:"); scanf("%s",des);
    fdsrc=open("/etc/passwd",O_RDONLY);
    if(fdsrc<0) { exit(1); }
    fddes=open(des,flags,0644);
    if(fddes<0) { exit(1); }
    while((nbytes=read(fdsrc,buf,20))>0)
    {
        z=write(fddes,buf,nbytes);
        if(z<0) { perror("写目标文件出错"); }
    }
    close(fddes);
    close(fdsrc);
    printf("复制\"/etc/passwd\"文件为\"%s\"文件成功! \n",des);
    exit(0);
}
```



## 5. lseek 函数 定位文件指针

- read/write 定位文件指针

```
#include <sys/types.h>
```

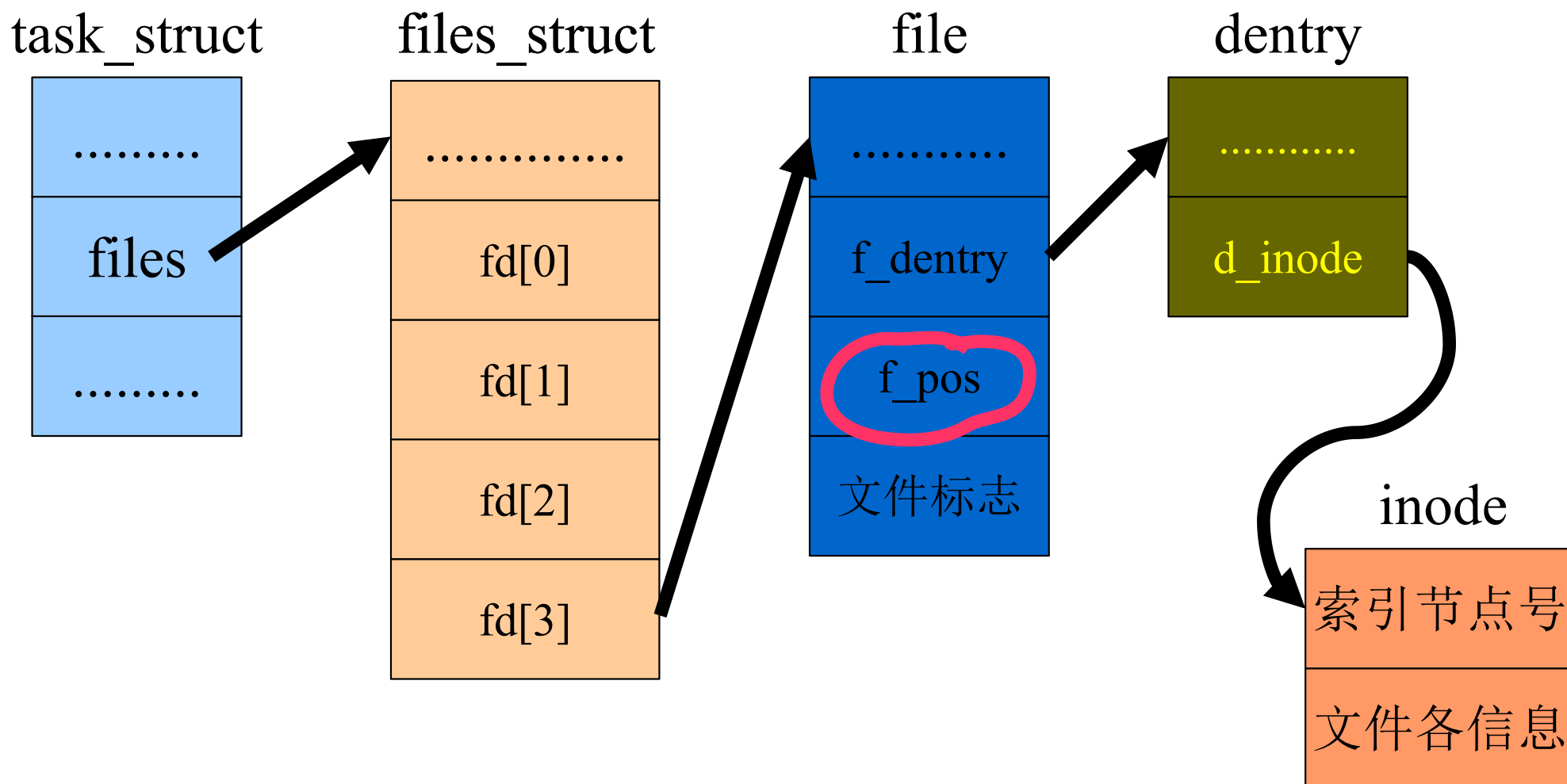
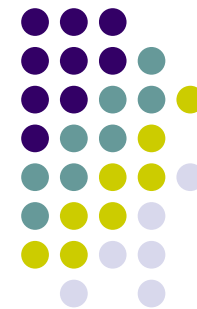
```
#include <unistd.h>
```

```
off_t lseek(int fd, off_t offset, int whence);
```

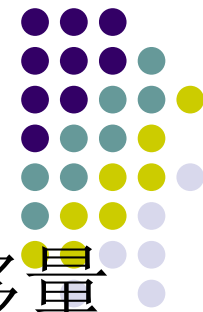
(Return: the resulting offset location if success; -1 if failure)

- The directive “whence”:
  - **SEEK\_SET**: the offset is set to “offset” bytes 指针位移量为设定值
  - **SEEK\_CUR**: the offset is set to its current location plus “offset” bytes 指针位移量为当前位移加设定值
  - **SEEK\_END**: the offset is set to the size of the file plus “offset” bytes 指针位移量为文件尾加设定值

## 5. lseek 函数



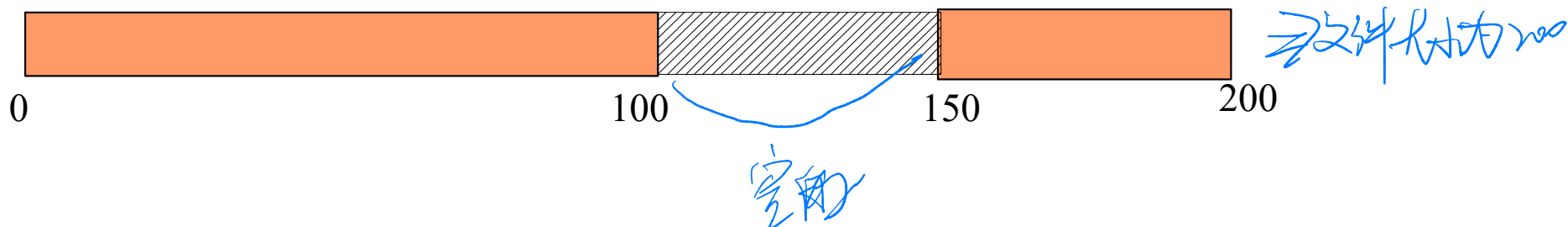
# 空洞文件

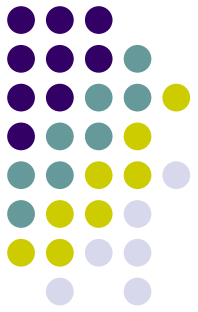


- 使用lseek修改文件偏移量后，当前文件偏移量有可能大于文件的长度
- 在这种情况下，对该文件的下一次写操作，将加长该文件
- 这样文件中形成了一个空洞。对空洞区域进行读，均返回0

```
lseek(fd, 50, SEEK_END)  
write(fd, buf, 50)
```

文件长度是多少？  
200? 150?





## 6. dup/dup2 函数

- 复制文件描述符

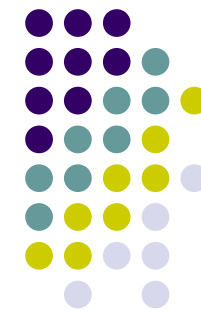
#include <unistd.h>

int dup(int oldfd);

int dup2(int oldfd, int newfd);

(Return: the new file descriptor if success; -1 if failure)

成功: 新的文件描述符



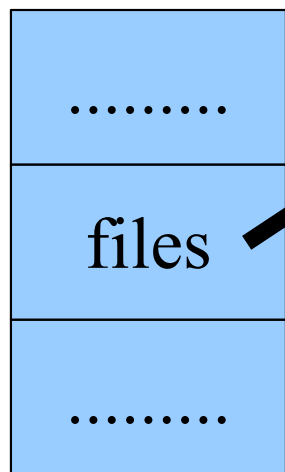
## 6. dup/dup2 函数

调用dup函数之前



*复制文件到*  
`int fd = dup(1);`  
*将1号文件描述符复制一下*

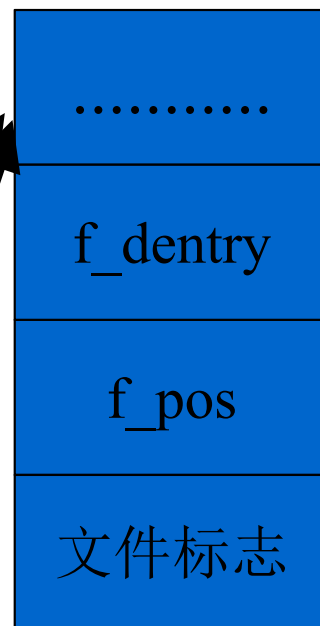
task\_struct



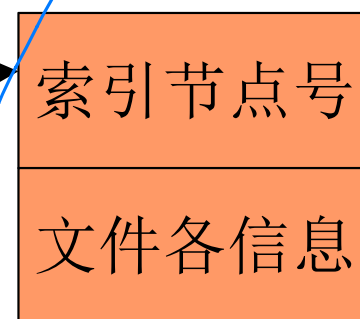
files\_struct



file



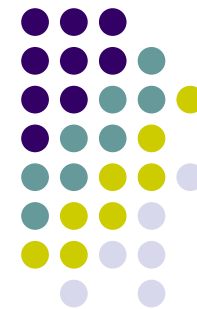
inode



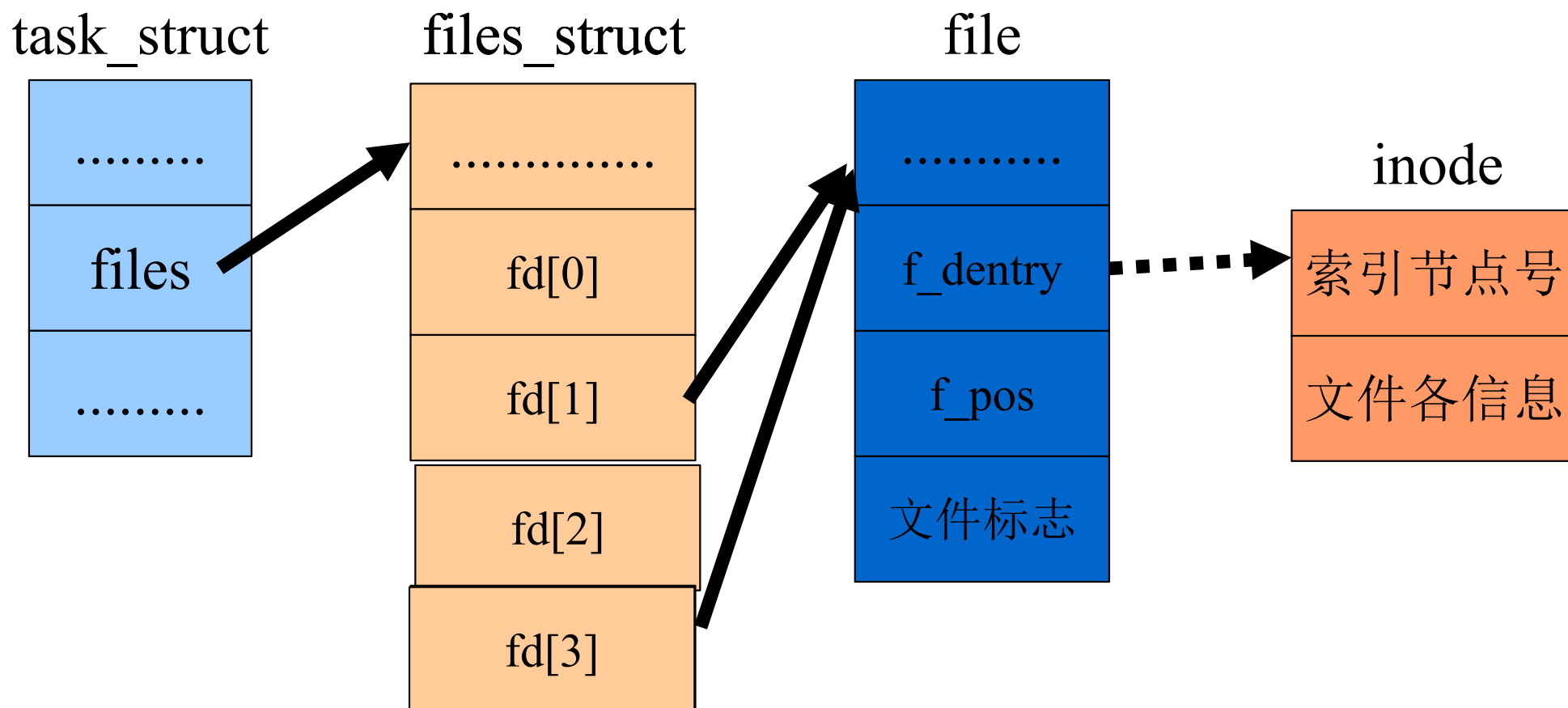
当前未占用文件描述符中的最小值

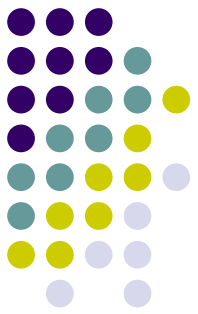
*也对应文件描述符*  
*输出文件*

## 6. dup/dup2 函数



使新老文件描述符，都指向  
同一个文件对象



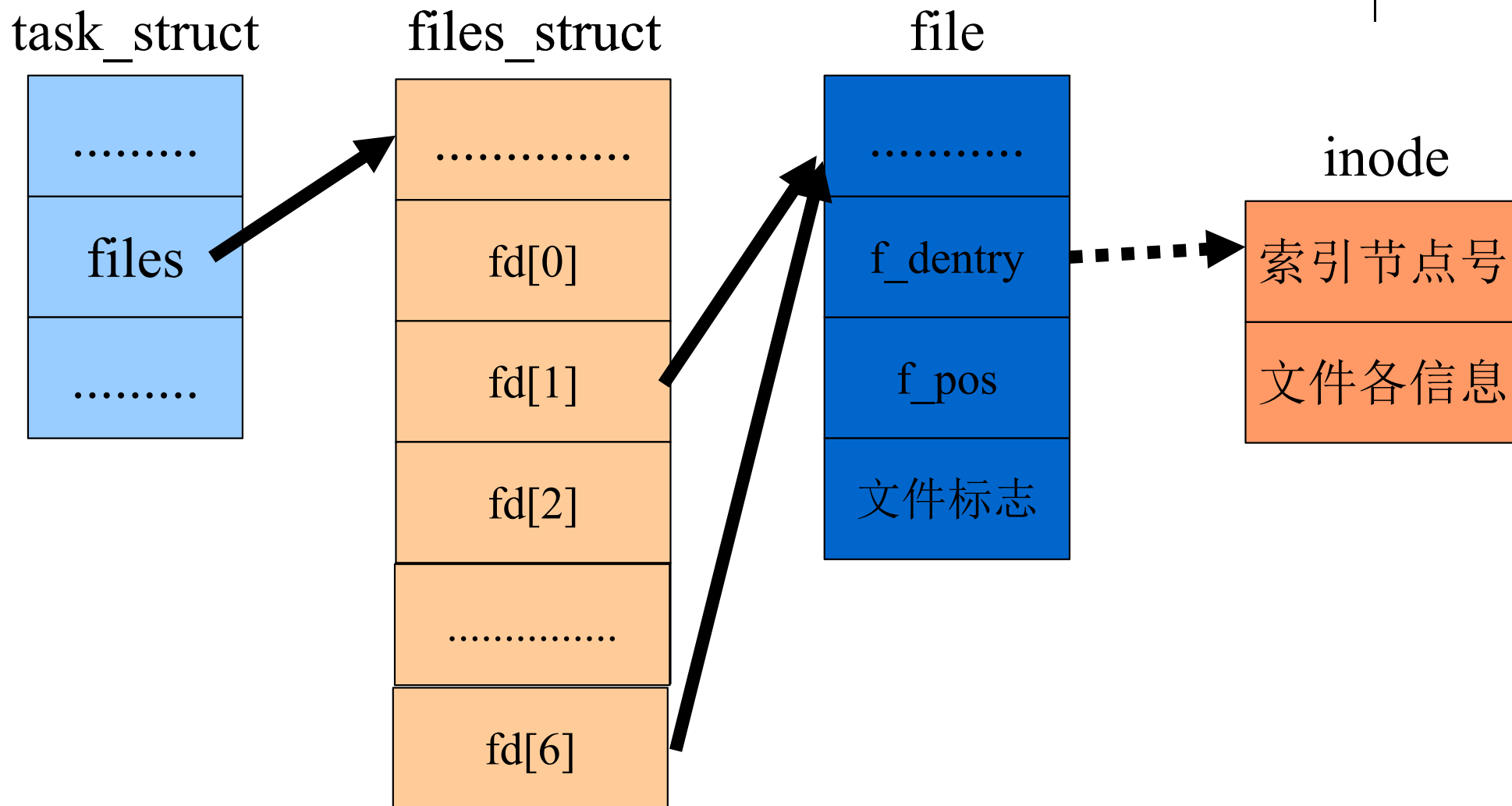
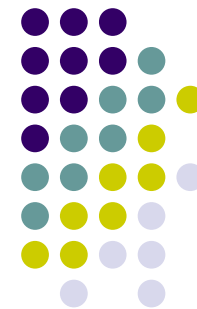


## 6. dup/dup2 函数

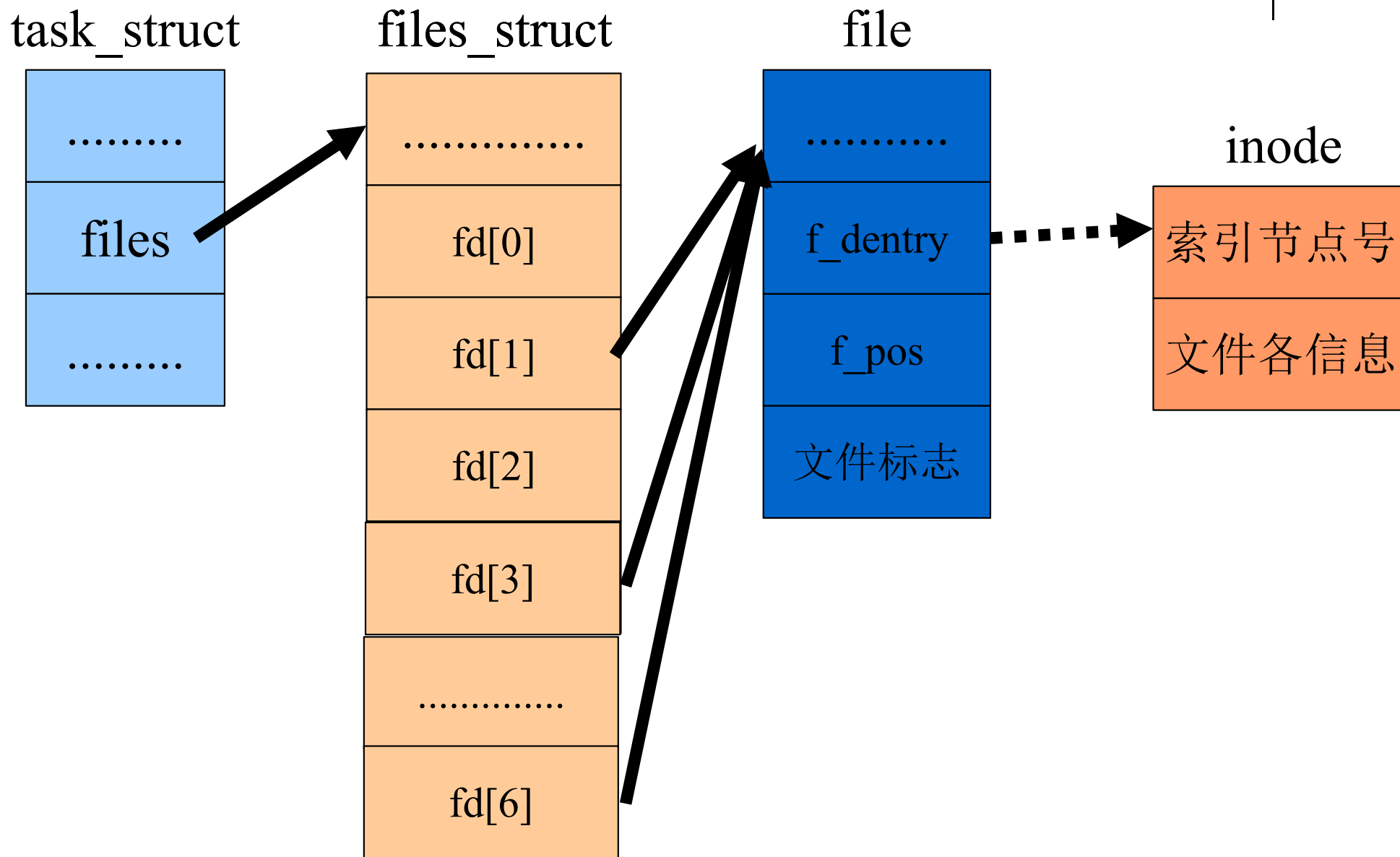
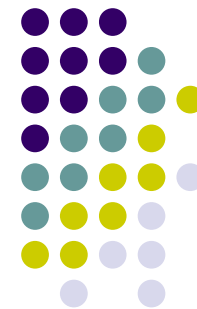
- 假设进程已打开文件描述符0、1、2
- 调用dup2(1, 6), dup2返回值是多少? 6
- 然后再调用dup(6), dup返回值是多少?

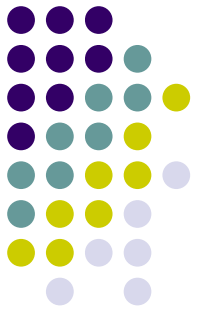
返回最小的 > 即 3

## 6. dup/dup2 函数



## 6. dup/dup2 函数





## 7. fcntl函数

- 查看、修改打开的文件描述符

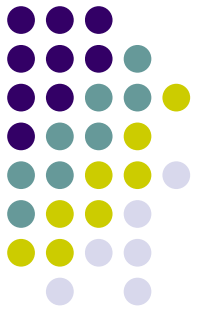
```
#include<fcntl.h>
```

```
#include<unistd.h>
```

```
#include<sys/types.h>
```

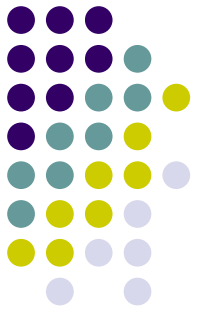
```
int result=fcntl(int fd,int cmd);
```

```
int result=fcntl(int fd,int cmd,long arg,...);
```



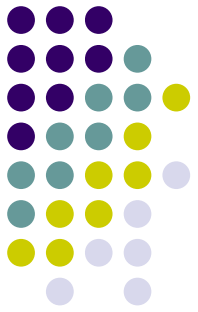
## 7. fcntl函数

- cmd取值:
  - ❖ F\_DUPFD 复制文件描述符
  - ❖ F\_GETFD 获得文件描述符
  - ❖ F\_SETFD 设置文件描述符
  - ❖ F\_GETFL 获取文件描述符当前模式
  - ❖ F\_SETFL 设置文件描述符当前模式
  - ❖ F\_GETLK 获得记录锁
  - ❖ F\_SETLK 设置记录锁



# 掌握的知识点

- 两种I/O方式区别
- 文件描述符概念
- 主要I/O函数：open，read/write，lseek，dup/dup2用法



# 课堂练习

## 分析以下程序运行情况

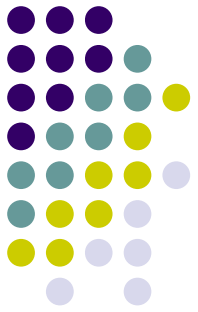
```
#include<sys/types.h>
#include<sys/stat.h>
#include<fcntl.h>
#include<stdio.h>
#include<stdlib.h>
char buf1[]="abcdefghij";
char buf2[]="ABCDEFGHGIJ";
main()
{int fd;
  if((fd=open("file.hole",O_WRONLY|O_CREAT/*|O_APPEND*/,0644))<0)
    perror("creat error");
  if(write(fd,buf1,10)!=10)
    perror("buf1 write error");
  if(lseek(fd,40,SEEK_SET)==-1)
    perror("lseek error");
  if(write(fd,buf2,10)!=10)
    perror("buf2 write error");
  exit(0);
}
```

(1) **ls -l** 查看文件大小多少字节?

(2) **cat** 查看文件内容?

(3) **od -c** 查看文件内容

51  
20-40为定, 10个字节  
40-50 ✓  
51  
10-40 ✓  
40-50 ✓



# 课堂练习

## 分析以下程序运行结果

```
#include<sys/types.h>
```

```
#include<sys/stat.h>
```

```
#include<fcntl.h>
```

```
#include<stdio.h>
```

```
#include<unistd.h>
```

```
main()
```

```
{ int fd;
```

```
if((fd=open("myout",O_WRONLY|O_CREAT,0644))!=-1)
```

```
    perror("myout open error");
```

```
if(dup2(fd,STDOUT_FILENO)!=-1)
```

```
    perror("redirect stand output failed");
```

```
printf("this is a test program for redirect\n");
```

```
close(fd);
```

```
}
```



(1) 查看程序运行结果?

(2) **cat** 查看文件**myout**的内容

fd为3

预定义的, 标准输出 → 1

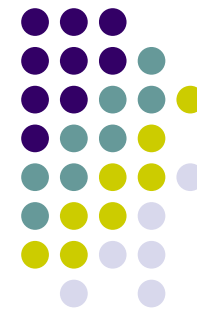
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 197, 198, 199, 200, 201, 202, 203, 204, 205, 206, 207, 208, 209, 210, 211, 212, 213, 214, 215, 216, 217, 218, 219, 220, 221, 222, 223, 224, 225, 226, 227, 228, 229, 230, 231, 232, 233, 234, 235, 236, 237, 238, 239, 240, 241, 242, 243, 244, 245, 246, 247, 248, 249, 250, 251, 252, 253, 254, 255, 256, 257, 258, 259, 260, 261, 262, 263, 264, 265, 266, 267, 268, 269, 270, 271, 272, 273, 274, 275, 276, 277, 278, 279, 280, 281, 282, 283, 284, 285, 286, 287, 288, 289, 290, 291, 292, 293, 294, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 315, 316, 317, 318, 319, 320, 321, 322, 323, 324, 325, 326, 327, 328, 329, 330, 331, 332, 333, 334, 335, 336, 337, 338, 339, 340, 341, 342, 343, 344, 345, 346, 347, 348, 349, 350, 351, 352, 353, 354, 355, 356, 357, 358, 359, 360, 361, 362, 363, 364, 365, 366, 367, 368, 369, 370, 371, 372, 373, 374, 375, 376, 377, 378, 379, 380, 381, 382, 383, 384, 385, 386, 387, 388, 389, 390, 391, 392, 393, 394, 395, 396, 397, 398, 399, 400, 401, 402, 403, 404, 405, 406, 407, 408, 409, 410, 411, 412, 413, 414, 415, 416, 417, 418, 419, 420, 421, 422, 423, 424, 425, 426, 427, 428, 429, 430, 431, 432, 433, 434, 435, 436, 437, 438, 439, 440, 441, 442, 443, 444, 445, 446, 447, 448, 449, 450, 451, 452, 453, 454, 455, 456, 457, 458, 459, 460, 461, 462, 463, 464, 465, 466, 467, 468, 469, 470, 471, 472, 473, 474, 475, 476, 477, 478, 479, 480, 481, 482, 483, 484, 485, 486, 487, 488, 489, 490, 491, 492, 493, 494, 495, 496, 497, 498, 499, 500, 501, 502, 503, 504, 505, 506, 507, 508, 509, 510, 511, 512, 513, 514, 515, 516, 517, 518, 519, 520, 521, 522, 523, 524, 525, 526, 527, 528, 529, 530, 531, 532, 533, 534, 535, 536, 537, 538, 539, 540, 541, 542, 543, 544, 545, 546, 547, 548, 549, 550, 551, 552, 553, 554, 555, 556, 557, 558, 559, 560, 561, 562, 563, 564, 565, 566, 567, 568, 569, 570, 571, 572, 573, 574, 575, 576, 577, 578, 579, 580, 581, 582, 583, 584, 585, 586, 587, 588, 589, 590, 591, 592, 593, 594, 595, 596, 597, 598, 599, 600, 601, 602, 603, 604, 605, 606, 607, 608, 609, 610, 611, 612, 613, 614, 615, 616, 617, 618, 619, 620, 621, 622, 623, 624, 625, 626, 627, 628, 629, 630, 631, 632, 633, 634, 635, 636, 637, 638, 639, 640, 641, 642, 643, 644, 645, 646, 647, 648, 649, 650, 651, 652, 653, 654, 655, 656, 657, 658, 659, 660, 661, 662, 663, 664, 665, 666, 667, 668, 669, 670, 671, 672, 673, 674, 675, 676, 677, 678, 679, 680, 681, 682, 683, 684, 685, 686, 687, 688, 689, 690, 691, 692, 693, 694, 695, 696, 697, 698, 699, 700, 701, 702, 703, 704, 705, 706, 707, 708, 709, 710, 711, 712, 713, 714, 715, 716, 717, 718, 719, 720, 721, 722, 723, 724, 725, 726, 727, 728, 729, 730, 731, 732, 733, 734, 735, 736, 737, 738, 739, 740, 741, 742, 743, 744, 745, 746, 747, 748, 749, 750, 751, 752, 753, 754, 755, 756, 757, 758, 759, 760, 761, 762, 763, 764, 765, 766, 767, 768, 769, 770, 771, 772, 773, 774, 775, 776, 777, 778, 779, 780, 781, 782, 783, 784, 785, 786, 787, 788, 789, 790, 791, 792, 793, 794, 795, 796, 797, 798, 799, 800, 801, 802, 803, 804, 805, 806, 807, 808, 809, 810, 811, 812, 813, 814, 815, 816, 817, 818, 819, 820, 821, 822, 823, 824, 825, 826, 827, 828, 829, 830, 831, 832, 833, 834, 835, 836, 837, 838, 839, 840, 841, 842, 843, 844, 845, 846, 847, 848, 849, 850, 851, 852, 853, 854, 855, 856, 857, 858, 859, 860, 861, 862, 863, 864, 865, 866, 867, 868, 869, 870, 871, 872, 873, 874, 875, 876, 877, 878, 879, 880, 881, 882, 883, 884, 885, 886, 887, 888, 889, 890, 891, 892, 893, 894, 895, 896, 897, 898, 899, 900, 901, 902, 903, 904, 905, 906, 907, 908, 909, 910, 911, 912, 913, 914, 915, 916, 917, 918, 919, 920, 921, 922, 923, 924, 925, 926, 927, 928, 929, 930, 931, 932, 933, 934, 935, 936, 937, 938, 939, 940, 941, 942, 943, 944, 945, 946, 947, 948, 949, 950, 951, 952, 953, 954, 955, 956, 957, 958, 959, 960, 961, 962, 963, 964, 965, 966, 967, 968, 969, 970, 971, 972, 973, 974, 975, 976, 977, 978, 979, 980, 981, 982, 983, 984, 985, 986, 987, 988, 989, 990, 991, 992, 993, 994, 995, 996, 997, 998, 999, 1000

↓  
向标准输出

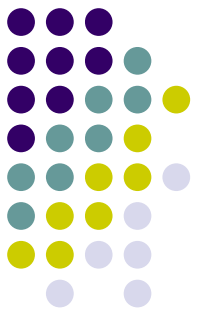
↓  
3和1对应 "myout"

1 → 向 myout

# Part 4 缓冲文件I/O

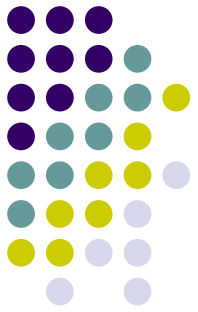


- 标准I/O库
- 文件流
- 标准I/O函数



## 4.1 标准I/O库

- 为什么要设计标准I/O库？
  - 直接使用API进行文件访问时，需要考虑许多细节问题
  - 例如：read、write时，缓冲区的大小该如何确定，才能使效率最优
- 标准I/O库封装了诸多细节问题，包括缓冲区分配



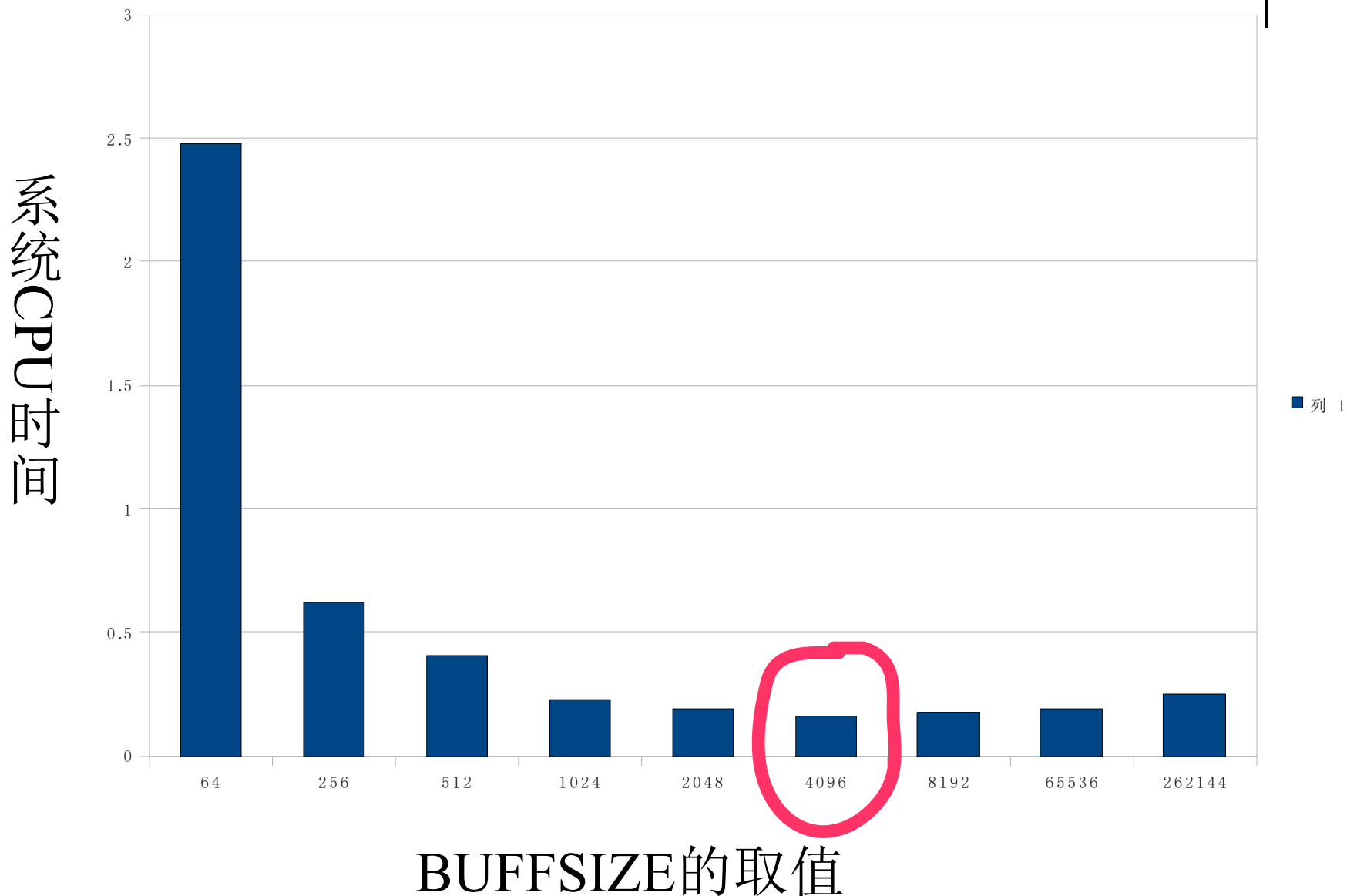
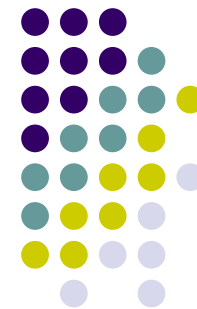
# 4.1 标准I/O库

## ■ I/O效率示例

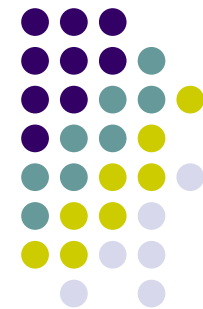
```
#define BUFSIZE 4096
#include<stdio.h>
#include<unistd.h>
int main()
{
    int n;
    char buf[BUFSIZE];
    while((n = read(STDIN_FILENO, buf, BUFSIZE))>0)
        if(write(STDOUT_FILENO, buf,n)!= n)
            perror("write error");
    return 0;
}
```

程序中，影响效率的关键：  
BUFSIZE的取值

## 4.1 标准I/O库——I/O效率

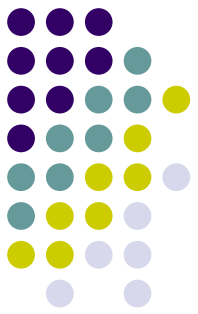


# 4.1 标准I/O库——I/O效率



## ■ 原因

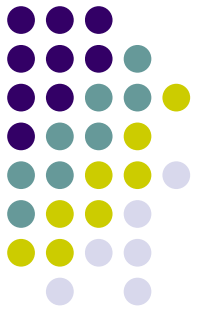
- Linux文件系统采用了某种预读技术
- 当检测到正在进行顺序读取时，系统就试图读入比应用程序所要求的更多数据
- 并假设应用程序很快就会读这些数据
- 当BUFSIZE增加到一定程度后，预读就停止了



## 4.1 标准I/O库

### 标准 I/O 缓冲

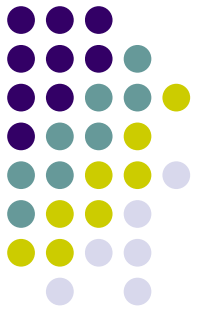
- 标准I/O库提供缓冲的目的：尽可能减少使用 `read`、`write`调用的次数，以提高I/O效率。
- 通过标准I/O库进行的读写操作，数据都会被放置在标准I/O库缓冲中中转。



## 4.2 文件流

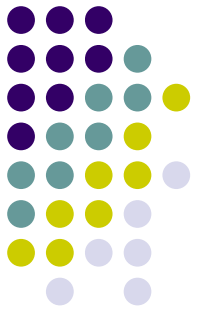
- Buffered I/O带缓冲I/O
- Stream and “FILE” structure
  - FILE\* fp;
  - 预定义的文件指针: stdin, stdout, stderr

## 4.2 文件流



标准I/O库  
管理的缓冲区

```
typedef struct {  
.....  
char      fd;           /* File descriptor      */  
short     bsize;        /* Buffer size          */  
unsigned char *buffer;  /* Data transfer buffer */  
.....  
} FILE;
```

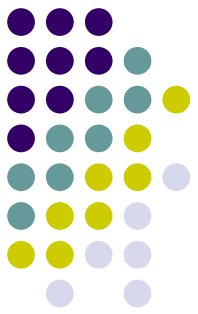


## 4.3 标准 I/O 函数

流

- 流 open/close
- 流 read/write
  - 每次一个字符的I/O: fgetc, fputc
  - 每次一行的I/O: fgets, fputs, gets, puts
  - 直接I/O(二进制I/O): fread, fwrite
  - 格式化I/O: scanf, printf, fscanf, fprintf
- 流定位: fseek, ftell, frewind
- 流刷新: fflush

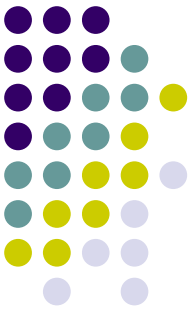
seek 不常调用



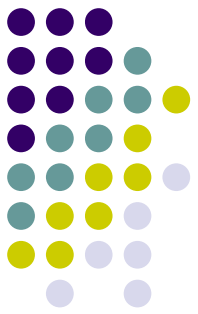
# 1. 流打开/关闭

- 例2.10设计一个程序，要求用流文件I/O操作打开文件“**2-10file**”，如果该文件不存在，则创建文件。

# Example2-10.c



```
#include<stdio.h>
#include<stdlib.h>
int main()
{
    FILE * fp;
    if((fp=fopen("./2-10file","a+"))==NULL)
    {
        printf("打开(创建)文件出错");    exit(0);
    }
    fclose(fp);
}
```



# 1.流打开/关闭

- Open a stream

`#include <stdio.h>`

`FILE *fopen(const char *filename, const char *mode);`

- Parameter “mode”

“r”: Open text file for reading.

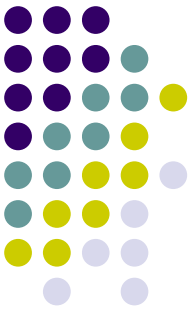
“w”: Truncate file to zero length or create text file for writing.

“a”: Open for appending.

“r+”: Open for reading and writing. 可读可写

“w+”: Open for reading and writing. The file is created if it does not exist, otherwise it is truncated. 可读可写, 文件不存在则创建, 存在则清空.

“a+”: Open for reading and appending. The file is created if does not exist.



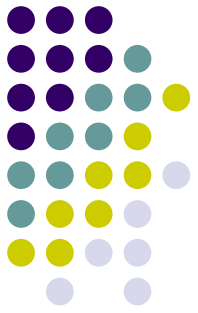
# 1.流打开/关闭

- Close a stream

```
#include <stdio.h>
```

```
int fclose(FILE *fp);
```

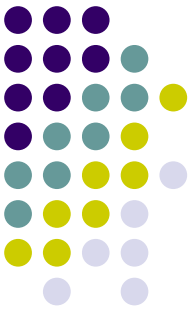
(Return: 0 if success; -1 if failure)



## 2.流读/写

对流有三种读写方式

- 每次读写一个字符 *goe*
- 每次读写一行
- 每次读写任意长度的内容



## (1) 输入/出一个字符

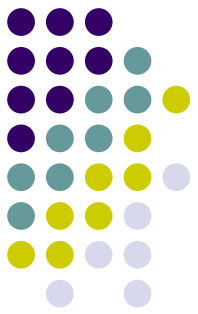
- getc, fgetc, getchar functions

```
#include <stdio.h>
```

```
int getc(FILE *fp);
```

```
int fgetc(FILE *fp);
```

```
int getchar(void);
```



## (1) 输入/出一个字符

- putc, fputc, putchar functions

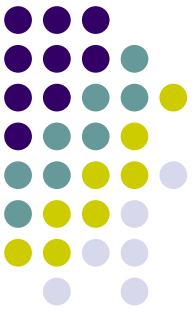
```
#include <stdio.h>
```

```
int putc(int c, FILE *fp);
```

```
int fputc(int c, FILE *fp);
```

```
int putchar(int c);
```

(Return: the character if success; -1 if failure)



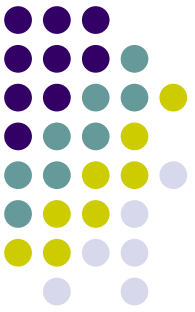
## (2) 输入/出一行

- fgets, gets functions

#include <stdio.h>

char \*fgets(char \*s, int size, FILE \*stream);

char \*gets(char \*s);



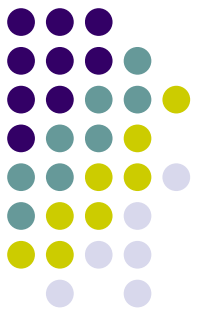
## (2) 输入/出一行

- fputs, puts functions

`#include <stdio.h>`

`int fputs(const char *s, FILE *stream);`

`int puts(const char *s);`



## (3) 二进制流输入/输出

- fread/fwrite functions

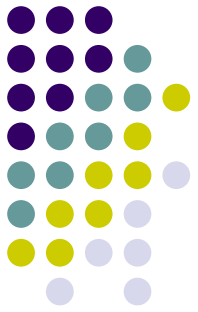
`#include <stdio.h>`

`size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream);`

`size_t fwrite(const void *ptr, size_t size, size_t nmemb, FILE *stream);`

(Return: the number of a items successfully read or written.)

- 例2.11设计两个程序，要求一个程序把三个人的姓名和账号余额信息通过一次流文件I/O操作写入文件“2-11file”，另一个格式输出账号信息，把每个人的账号和余额一一对应显示输出。



# Example 2-11 fread.c

*/\*程序：把帐号信息从文件读出\*/*

```
#include<stdio.h>
```

```
#define nmemb 3
```

```
struct test
```

```
{ char name[20];
```

```
    int pay;
```

```
}s[nmemb];
```

```
int main( )
```

```
{ FILE * fp;
```

```
    int i;
```

```
    fp = fopen("2-11file", "r");
```

```
    fread(s,sizeof(struct test),nmemb,fp);
```

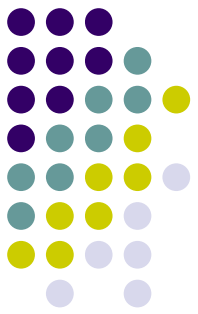
```
    fclose(fp);
```

```
    for(i=0;i<nmemb;i++)
```

```
        printf("帐号[%d]:%-20s余额[%d]:%d\n",i,s[i].name,i,s[i].pay);
```

```
    return 0;
```

```
}
```



# Example 2-11 fwrite.c

/\*程序：把帐号信息写入文件\*/

```
#include<stdio.h>
```

```
#include<string.h>
```

```
#define set_s(x,y,z) {strcpy(s[x].name,y);s[x].pay=z;} /*自定义宏，用于赋值*/
```

```
#define nmemb 3
```

```
struct test
```

```
{   char name[20];
```

```
    int pay;
```

```
}s[nmemb];
```

```
int main()
```

```
{   FILE * fp;
```

```
    set_s(0,"张三",12345);
```

```
    set_s(1,"李四",200);
```

```
    set_s(2,"王五",50000);
```

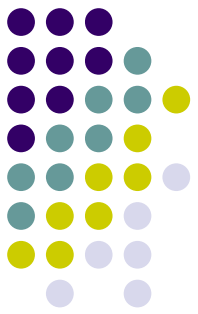
```
    fp=fopen("2-11file","a+");          /*打开(创建)文件*/
```

```
    fwrite(s,sizeof(struct test),nmemb,fp);
```

```
    fclose(fp);
```

```
    return 0;
```

```
}
```



## (4) 格式化 I/O

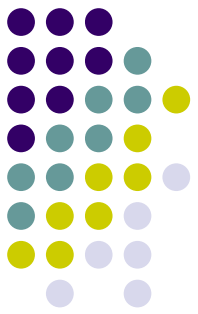
- scanf, fscanf, sscanf functions

```
#include <stdio.h>
```

```
int scanf(const char *format, ...);
```

```
int fscanf(FILE *stream, const char *format, ...);
```

```
int sscanf(const char *str, const char *format, ...);
```



## (4) 格式化 I/O

- printf, fprintf, sprintf functions

#include <stdio.h>

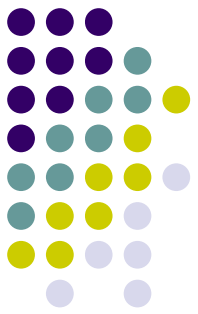
int printf(const char \*format, ...);

int fprintf(FILE \*stream, const char \*format, ...);

int sprintf(char \*str, const char \*format, ...);

→ 指定写入哪个文件

printf 格式化输出



### 3. 流定位

为系统指针函数。

- **fseek, ftell, rewind functions**

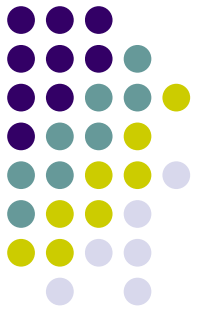
```
#include <stdio.h>
```

```
int fseek(FILE *stream, long int offset, int  
whence);
```

```
long ftell(FILE *stream);
```

```
void rewind(FILE *stream);
```

- **例2.12** 设计一程序，要求用**fopen**函数打开系统文件“/etc/passwd”，先把位置指针移动到第**10**个字符前，再把位置指针移动到文件尾，最后把位置指针移动到文件头，输出三次定位的文件偏移量的值。



# Example 2-12.c

/\*程序：文件的定位\*/

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    FILE *stream;
```

```
    long offset;
```

```
    fpos_t pos;
```

```
    stream=fopen("/etc/passwd","r");
```

```
    fseek(stream,10,SEEK_SET);
```

```
    printf("文件流的偏移量: %d\n",ftell(stream));
```

```
    fseek(stream,0,SEEK_END); 文件尾+10
```

```
    printf("文件流的偏移量: %d\n",ftell(stream));
```

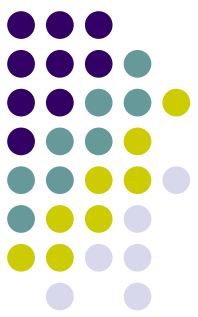
```
    rewind(stream); 回到开头
```

```
    printf("文件流的偏移量: %d\n",ftell(stream));
```

```
    fclose(stream);
```

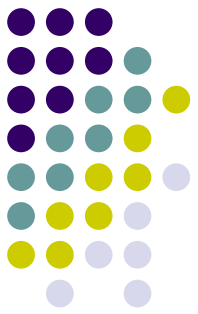
```
    return 0;
```

```
}
```



# 运行结果

```
mylinux@mylinux-desktop: ~/code/ch2
File Edit View Terminal Tabs Help
mylinux@mylinux-desktop: ~/code/ch2$ gcc -o 2-13 2-13.c
mylinux@mylinux-desktop: ~/code/ch2$ ./2-13
文件流的偏移量 : 10
文件流的偏移量 : 1495
文件流的偏移量 : 0
mylinux@mylinux-desktop: ~/code/ch2$
```



## 4. 流刷新

- 刷新文件流。把流里的数据立刻写入文件——**fork前使用**

**fflush**

→ 强制刷新

```
#include <stdio.h>
```

```
int fflush(FILE *stream);
```

- 自动刷新:

① 流关闭 **fclose**;

② **exit** 终止;

③ 行缓冲 “\n”;

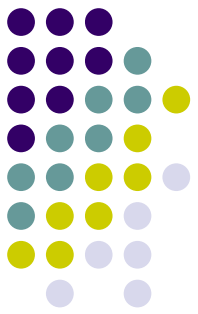
④ 缓冲区满;

⑤ 执行输入操作读文件: **printf** (“hello”) ; **scanf** (“%d”, &a)

printf 缓冲区 行为标记 , 加 \n → 刷新

scanf 缓冲区

先刷新缓冲区



## 5. 流缓冲

- 三种类型的缓冲

- 块（全）缓冲 **block buffered (fully buffered)**

：一般C库函数写入文件是全缓冲的 *填满后刷新*

- 行缓冲 **line buffered**: 引用标准交互设备的流 *行*  
**stdin, stdout**

例： `for(i=1;i<=10;i++) fputc(c, stdout);`  
`fputc("\n", stdout);`

- 无缓冲 **Unbuffered**: 标准错误流 **stderr**

# 5. 流缓冲



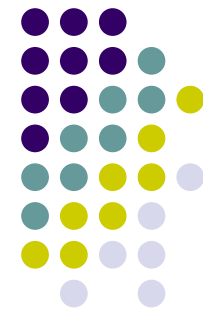
## ■ 全缓冲

- 在填满标准I/O缓冲区后，才进行实际I/O操作（例如调用write函数）
- 调用fflush函数也能强制进行实际I/O操作

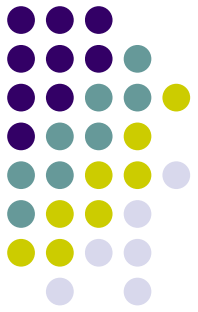
## ■ 行缓冲

- 在输入和输出遇到换行符时，标准I/O库执行I/O操作
- 因为标准I/O库用来收集每一行的缓存的长度是固定的，所以，只要填满了缓存，即使没有遇到新行符，也进行I/O操作
- 终端（例如标准输入和标准输出），使用行缓冲

# 5. 流缓冲



- 不带缓冲
  - 标准I/O库不对字符进行缓冲存储
  - 标准出错是不带缓冲的，为了让出错信息尽快显示出来



# 流和文件描述符

- 确定流使用的底层文件描述符

```
#include <stdio.h>
```

```
int fileno(FILE *fp);
```

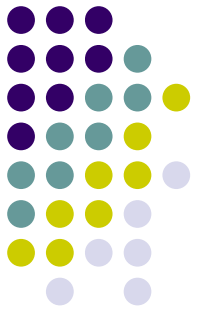
- 根据已打开的文件描述符创建一个流

```
#include <stdio.h>
```

```
FILE *fdopen(int fildes, const char *mode);
```

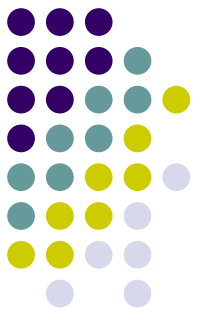
返回文件流指针。

way



# 掌握的知识点

- 标准I/O库的作用
- `fopen`函数的使用
- 三种缓冲
- 流和文件描述符转换



# 本章总结

1. 文件属性
  - 文件属性操作
2. Basic file I/O
  - 基本I/O系统调用(open, close, lseek, read, write, dup/dup2...)
3. 文件和目录
  - 系统调用stat/fstat/lstat; opendir, readdir, ...
4. Standard I/O library: fopen, fflush